STUDYSPACE: A COLLABORATIVE WEB APPLICATION FOR ACADEMIC STUDENTS

Thet Oo Aung

Bachelor's thesis
Faculty of Information Technology
Czech Technical University in Prague
Department of Software Engineering
Study program: Informatics
Specialisation: Software Engineering
Supervisor: Ing. Jan Bliznicenko, Ph.D.
June 28, 2026

FACULTY OF INFORMATION TECHNOLOGY CTU IN PRAGUE

Assignment of bachelor's thesis

Title:	StudySpace: A Collaborative Web Application for Academic Students
Student:	Thet Oo Aung
Supervisor:	Ing. Jan Bliznicenko, Ph.D.
Study program:	Informatics
Branch / specialization:	Software Engineering 2021
Department:	Department of Software Engineering
Validity:	until the end of summer semester 2026/2027

Instructions

The goal of this thesis is to design and implement a functional prototype of a web application that combines course materials management and student communication into a single environment.
The application aims to reduce the need for switching between different tools by linking study materials directly to peer discussions.

Key Functional Requirements:
1. Course Administration Module: An interface for instructors to create course structures, upload lecture materials, and handle student enrollments.
2. Content Extension System: This feature allows students to clone course materials into private workspaces for personal note-taking and group collaboration, or to propose their modifications to the instructor. The instructor can manually review them and choose to showcase the student's extended version alongside the official content for the entire class.
3. Contextual Messaging System: A communication tool that allows students to "anchor" questions to specific course materials. By typing a shortcut symbol (such as @), students can select a PDF or files from the course materials to include in their message. This creates a clickable link, allowing others to jump directly to the selected document for quick reference and support.
4. AI-Based Content Querying: Integration of the APIs from Large Language Models (e.g. OpenAI, Gemini) to allow users to query summaries or explanations based on the

---

Electronically approved by Ing. Michal Valenta, Ph.D. on 11 February 2026 in Prague.

context of the loaded study materials.
5. Access Control: A secure authentication/ authorization system to differentiate permissions depending on the roles: eg. Students (view/copy/discuss) and Instructors (manage content/approve merges).

Follow these steps:
1. Analyze Requirements: Define the functional requirements for StudySpace and analyze the specific needs of the target academic audience through user research.
2. Current Solutions Analysis: Research existing solutions (Learning Management Systems like Moodle, communication tools like Discord/Teams) to identify integration gaps and opportunities for improvement.
3. UI Design: Create wireframes to visualize the prototype's flow for a better user experience.
4. System Architecture: Design the database schema and API structure. Research and select the appropriate technology stack (programming languages, frameworks, libraries, tools), specifically addressing the data model required for personal content copies, merge proposals and contextual message anchoring.
5. Implementation: Prioritize the development of a functional prototype of the application with the frontend and backend components, integration of the LLM API, and implementation of all core features.
6. Testing: Perform functional and usability testing to ensure system stability and completeness of the core features.
7. Documentation: Write technical documentation for deployment and user guides for both students and instructors.

I would like to thank my supervisor, Ing. Jan Blizničenko, Ph.D., for his guidance throughout the preparation of this thesis. I would also like to thank my family for their continuous support and encouragement.

Declaration

I hereby declare that this bachelor's thesis entitled "StudySpace" was written by me independently and that I have used only the sources listed in the bibliography. I declare that I have not submitted this thesis or any part thereof for the purpose of obtaining any other degree or qualification.

I acknowledge that my thesis is subject to the Act No. 121/2000 Coll., the Copyright Act of the Czech Republic, and that Czech Technical University in Prague has the right to conclude a license agreement on the use of this thesis as specified by this Act.

In Prague on June 28, 2026

Abstract

This bachelor's thesis describes the design and implementation of a prototype of StudySpace, a web application that allows students and instructors to manage course materials and collaborate in one place. The application addresses the problem of students having to switch between multiple separate tools for materials, communication, and assignments.

The thesis presents an analysis of existing learning management systems and system requirements, followed by the design and implementation of key features: course management, a system where students can clone a workspace, add their own notes, and contribute the updated document back to the original course page for the instructor to review, messages with direct links to named course documents, and an AI assistant for querying study materials. The backend is built with Java and Spring Boot, the frontend with Next.js and React, using PostgreSQL for data storage. Deployment is automated via CI/CD pipelines to modern cloud platforms (Vercel, Render). The system is verified through unit and integration tests at both the service and API levels.

Keywords web application, collaboration, React, Spring Boot, Next.js, PostgreSQL, REST API, CI/CD, WebSocket

Abstrakt

Tato bakalářská práce popisuje návrh a implementaci prototypu webové aplikace StudySpace, která umožňuje studentům a vyučujícím spravovat studijní materiály a spolupracovat v jednom prostředí. Aplikace řeší problém, kdy studenti dnes používají několik různých nástrojů najednou pro materiály, komunikaci i odevzdávání úkolů.

Práce zahrnuje analýzu existujících systémů pro správu výuky a následnou implementaci klíčových funkcí: správu kurzů, systém, ve kterém si student může naklonovat studijní pracovní prostor, přidat vlastní poznámky a upravený dokument přispět zpět na původní stránku kurzu ke kontrole vyučujícím, zasílání zpráv s přímým odkazem na konkrétní studijní materiál a dotazování materiálů pomocí umělé inteligence. Backend je postaven na Java a Spring Boot, frontend na Next.js a React, databáze je PostgreSQL. Nasazení využívá moderní CI/CD pipelines do cloudového prostředí (Vercel, Render). Systém je otestován jednotkovými a integračními testy na úrovni služeb i API.

Klíčová slova webová aplikace, spolupráce, React, Spring Boot, Next.js, PostgreSQL, REST API, CI/CD, WebSocket

Contents

List of Figures

List of Tables

List of Code listings

List of abbreviations

ACID	Atomicity, Consistency, Isolation, Durability
AI	Artificial Intelligence
API	Application Programming Interface
CS	Computer Science
DB	Database
DTO	Data Transfer Object
ER	Entity-Relationship
ESM	ECMAScript Modules
HTTP	Hypertext Transfer Protocol
HMR	Hot Module Replacement
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
LMS	Learning Management System
MVC	Model-View-Controller
ORM	Object-Relational Mapping
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single Page Application
SQL	Structured Query Language
SSR	Server-Side Rendering
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience
UML	Unified Modeling Language

Chapter 1

Introduction

Imagine a student preparing for an exam at the Faculty of Information Technology at CTU in Prague. He has a question about a diagram in the lecture notes, but to ask his peers he must switch to Discord, manually reference the file, and hope classmates locate the same slide in time. By the next morning the question has floated away in chat history, disconnected from the material it concerned and invisible to students who join the course the following semester. This scenario illustrates the fundamental problem that StudySpace is designed to address: the fragmentation of academic tools that forces students and instructors to operate across multiple disconnected platforms.

1.1 Motivation

Modern higher education relies on a combination of platforms that rarely communicate with one another. At a typical faculty, course materials reside in a learning management system such as Moodle or the faculty course portal, while student collaboration happens informally on Discord, WhatsApp, or Microsoft Teams. Code assignments and grading are handled by yet another system, and AI tools such as ChatGPT are used in isolation without any link to the actual curriculum content. The consequence of this fragmentation is that information ends up scattered: a useful correction that a student discovers during self-study has no path back into the course, a question about a specific document carries no reference to the document itself, and knowledge generated by one cohort is invisible to the next.

StudySpace addresses these gaps by combining four capabilities in a single environment. Instructors need a structured way to organise and publish course materials and to manage which students can access them (F1). Students, rather than passively consuming that content, need a mechanism to clone a workspace, enrich it with their own notes, and contribute improvements back to the original course page for the instructor to review and optionally publish (F2).

When students want to ask a question about a specific document, they need a messaging channel that can attach a direct reference to that document, so other readers can follow the link and land on the relevant material immediately (F3). Finally, students need an AI assistant that queries the actual course content rather than operating on general knowledge alone, grounding answers in the material the instructor has provided (F4). All of these capabilities require a role-based access model that distinguishes instructors from students at the API level, preventing students from modifying course materials they do not own (F5).

1.2 Structure

Chapter 2 defines the target audience and their pain points, establishes the functional and non-functional requirements derived from the motivation above, examines existing platforms and their limitations, and describes the planned system and logical architecture. Chapter 3 translates the analysis into concrete technical decisions, covering the domain model, the technology choices and the rationale behind each, the database schema, and the user interface design. Chapter 4 documents the implementation of the five core features, presents the code listings that illustrate the most important design choices, and describes how the system was tested. Chapter 5 summarises the outcomes relative to the goals stated in this chapter, reflects on the known limitations of the prototype, and proposes directions for future work.

Chapter 2
Analysis

This chapter establishes the foundation for all subsequent design and implementation decisions. It begins by defining the problem and the target audience whose needs motivated this project. This is followed by an examination of existing platforms to identify the integration gap that none of them currently fills. A use-case view grouped by actor is then presented to illustrate system interactions, from which the formal functional and non-functional requirements are derived. Finally, the chapter justifies the major technology choices by evaluating available alternatives against these established requirements.

2.1 Problem Statement and Target Audience

The primary target audience of StudySpace consists of two groups whose workflows are closely coupled: *students* and *instructors* at a higher-education faculty. Students are the consumers and potential contributors of course content. Their typical lifecycle within a single course involves browsing uploaded materials, asking questions about specific documents, annotating or extending study notes independently, and occasionally contributing improvements that could benefit their peers. Instructors are the publishers and curators of that content. Their lifecycle involves creating a course structure, uploading materials into named sections, managing which students are enrolled, and reviewing any student contributions before deciding whether to make them visible to the whole class.

The Faculty of Information Technology at CTU in Prague serves as the representative context. A student's daily workflow involves switching between multiple systems: the faculty course portal or Moodle for lecture slides and reading assignments, Microsoft Teams or SharePoint for live sessions and recordings, Progtest or GitLab for programming assignments and grading, and Discord, WhatsApp, or email for informal peer discussion. This constant context-switching leads to scattered information, loss of peer-generated knowl-

edge, and extra effort spent locating the right tool for a given task. Discussions about a specific lecture slide happen in a general chat channel with no link to the actual material. Student notes and annotations remain in personal files and never contribute back to the course. There is no structured way for a student to propose improvements to lecture materials, and AI tools are used in isolation without any connection to the course content.

StudySpace aims to unify course management, student collaboration, contextual discussion, and AI assistance into a single platform, so that the interactions between these activities are preserved and available to all participants.

2.2 Existing Solutions

Several platforms address parts of the problem described in section 2.1. This section compares the most relevant ones against the core features that StudySpace provides, concluding with a synthesis of the integration gap that none of them fills end-to-end.

Table 2.1 Comparison of existing platforms against StudySpace features

Platform	Structured course materials	Student content contribution	Contextual discussion	AI content assistance	Role-based access control
Zoom / Teams	None	None	None	None	Partial
Discord	None	None	None	None	Partial
Moodle	Full	Limited	None	None	Full
Notion / Docs	Full	Partial	None	Generic	None
StudySpace	Full	Full	Full	Context	Full

The comparison in Table 2.1 shows that no existing platform covers all areas simultaneously. The following subsections analyse each platform in detail.

2.2.1 Zoom and Microsoft Teams

Zoom and Microsoft Teams are the dominant platforms for synchronous online education. Both support high-quality video conferencing, screen sharing, breakout rooms, and session recording. Microsoft Teams adds persistent channels, a file-sharing area backed by SharePoint, and deep integration with the Microsoft 365 productivity suite.

Despite these strengths, neither platform is designed for asynchronous, material-centric collaboration. Files shared during a meeting are stored in

a flat SharePoint folder rather than being organised into a course structure that reflects the academic curriculum. Discussion threads are tied to meeting events, making conversations about a lecture slide inaccessible to students who join the course later. There is no mechanism for students to annotate shared files and propose changes back to the Instructor; the Instructor remains the sole publisher of content. The asynchronous knowledge-management and student contribution use cases are left entirely unaddressed.

2.2.2 Discord

Discord was originally designed for gaming communities but has been widely adopted by student groups as an informal alternative to institutional platforms. Its server and channel model allows students to self-organise into topic-specific channels, and its low barrier to entry means that most students already have an account.

However, Discord's design is fundamentally chat-centric and lacks any concept of structured educational content. Files are posted as raw message attachments in a general channel with no link to a course section or curriculum structure. There is no role hierarchy that distinguishes Instructors from students at the permission level beyond basic moderation roles; an Instructor cannot publish a set of Course Materials in a read-only area that students can then clone and annotate. Content disappears into scroll history, there is no access control aligned with course enrollment, and there is no mechanism for structured content contribution or AI-assisted querying of Course Materials.

2.2.3 Moodle

Moodle is a mature, open-source Learning Management System (LMS) used by thousands of universities worldwide, including CTU. It provides the most comprehensive course management feature set of any platform considered: Instructors can define courses with hierarchical sections, upload files and links, set up graded assignments and quizzes, track student progress, and manage enrollment through a role-based permission model.

Despite this breadth, Moodle has well-documented limitations with respect to student agency and contextual collaboration. Its discussion model is based on forums that are standalone modules attached to a course but not anchored to specific Course Materials; a student cannot attach a comment directly to a particular document without navigating to a separate forum module. Student contribution in Moodle is limited to wiki modules and assignment submissions, which are graded artefacts that do not become part of the course content visible to other students. Moodle also has no built-in AI assistant that operates on the actual course content in real time.

2.2.4 Notion and Google Docs

Notion and Google Docs represent a different category: general-purpose collaborative writing tools. Both support real-time co-editing, rich text formatting, embedded media, and comment threads. Notion extends this with a hierarchical page structure and customisable database views, making it possible to approximate a course workspace.

The critical gaps are the absence of role-based access control aligned with academic roles and the lack of a structured contribution workflow. In a shared Notion workspace, any collaborator can edit any page, creating conflicts and the risk of accidental data loss in a course setting with many students. Google Docs has stricter permission levels but no multi-tier workflow where a student creates a private draft, develops it independently, and then submits it for review and publication. Neither platform is AI-aware in a course-content sense: Notion AI and Google's Gemini integration can generate or summarise text, but they operate on the open document in isolation, without knowledge of which course the document belongs to or what other materials are in the same section.

2.2.5 Summary of Gaps

The analysis above reveals a consistent pattern across all evaluated platforms. None of them provides a mechanism for anchoring a discussion message to a named Course Material within a structured course hierarchy, meaning that contextual references must be described manually in text and the connection to the source material is easily lost. No platform supports a clone-then-propose contribution model in which a student creates a private enriched copy of a course section and submits it back for Instructor review and publication; the Instructor remains the sole publisher of course content in every platform examined. Where AI assistance is available, as in Notion AI or Google's Gemini integration, it operates on the open document in isolation and cannot be grounded in the broader course context. The result is that satisfying all four requirements simultaneously forces students to use at least two to three separate platforms, reproducing the context-switching overhead described in section 2.1.

StudySpace addresses this integration gap by combining structured course management, a clone-based student contribution workflow with Instructor review, chat messages with Contextual Anchors tied to specific Course Materials, and an AI assistant grounded in the actual course content, all within a single application with role-based access control.

2.3 Use Cases

The system has two types of actors: *Student* and *Instructor*. The use cases below describe the main interactions each actor has with the system, grouped by

role. Pre-conditions are stated for the non-trivial cases that have dependencies on prior system state.

Instructor Use Cases

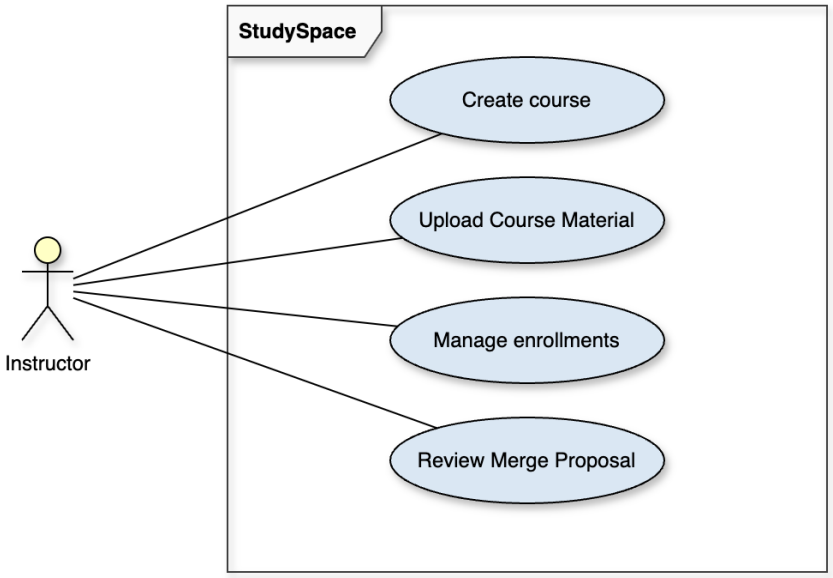

Figure 2.1 UML use case diagram for Instructor

- `Create course`: The Instructor creates a course with a title and description and sets its published status.
- `Upload Course Material`: Pre-condition: the course and at least one section exist. The Instructor selects a file, assigns a title, and uploads it into a named section. The backend infers the file type and persists the file URL.
- `Manage enrollments`: The Instructor approves or drops individual student enrollments from the course management panel.
- `Review Merge Proposal`: Pre-condition: at least one student has submitted a Merge Proposal for this course. The Instructor inspects the proposed Course Material and either approves it (publishing it on the main course page with the contributor's name attached) or rejects it with an optional review note.

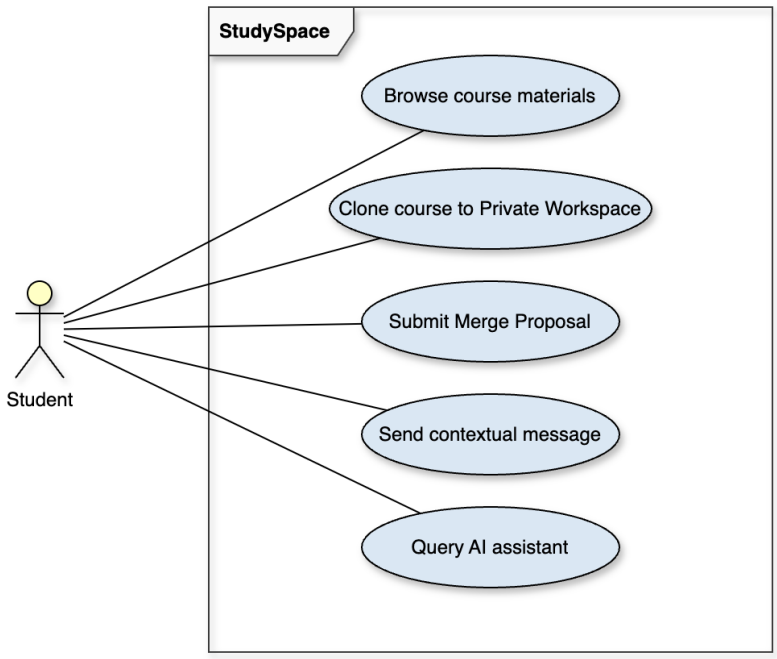

Figure 2.2 UML use case diagram for Student

Student Use Cases

- **`Browse course materials`**: The student navigates to an enrolled course and opens individual sections to read the uploaded Course Materials.
- **`Clone course to Private Workspace`**: Pre-condition: the student is enrolled in a published course. The student triggers a clone action; the backend creates a Private Workspace containing mirror copies of all sections and Course Materials.
- **Submit Merge Proposal**: Pre-condition: the student owns a Private Workspace with at least one non-reference material or a modified reference material. The student selects materials and a target section, optionally adds a message, and submits the proposal for Instructor review.
- **`Sent contextual message`**: The student types a message in the workspace chat, uses `@` to attach a Contextual Anchor to a specific Course Material, and submits. All enrolled users can see the message and follow the anchor.
- **`Query AI assistant`**: The student opens the AI panel, selects a Course Material for context, and types a question. The system sends the question and the extracted document text to the LLM API and displays the answer.

2.4 Functional Requirements

The following requirements are derived directly from the problem statement (section 2.1), the use cases (section 2.3), and the identified gaps in existing solutions (section 2.2). Each requirement is identified by an ID that is referenced throughout the remainder of the thesis.

- **FR-01 (Course Administration Module)**: An Instructor must be able to create, update, and delete courses and organise their content into named sections, while a Student must be able to browse and read any course they are enrolled in. The Instructor controls whether a course is published and manages enrollment per student. Course Materials (PDF documents, slide sets, video files, or images) are grouped into named sections, and the appropriate viewer is selected automatically based on the file type.
- **FR-02 (Content Extension System)**: A Student must be able to clone a course into a Private Workspace, add or modify materials independently, and submit a Merge Proposal for the Instructor to review and optionally publish. The clone operation creates a private mirror of the course structure that the student may enrich with additional materials. A submitted Merge Proposal enters an Instructor review queue where it can be approved and published back to the course, crediting the contributing student.
- **FR-03 (Contextual Messaging System)**: Users must be able to post messages in the workspace chat and attach a direct reference to a specific Course Material, so that other users can open the material from within the message. The Contextual Anchor is embedded in the message as a clickable badge and persists in the chat history, remaining accessible to all enrolled students regardless of when they join the course.
- **FR-04 (AI-Based Content Querying)**: Users must be able to ask natural-language questions about the content of the currently open workspace and receive answers grounded in that content. The AI assistant operates on the uploaded Course Materials rather than on general knowledge, and maintains conversation history so that follow-up questions can reference earlier turns within the same session.
- **FR-05 (Access Control)**: The system must recognise two roles, Student and Instructor, and enforce role-based restrictions at the API level so that students cannot perform instructor-only operations. Role assignment occurs at registration and is enforced on every request through token-based authentication. A student attempting an Instructor-only operation receives an authorisation error without the request reaching the business logic layer.

2.5 Non-Functional Requirements

The following non-functional requirements define the quality constraints the system must satisfy regardless of the specific feature being used.

- **NFR-01 (Usability)**: The interface should be straightforward enough for students with no prior training to use without a manual.
- **NFR-02 (Security)**: All endpoints except login and registration must require a valid JWT token. Role-based access must be enforced on the server side.
- **NFR-03 (Performance)**: Course and workspace pages should load within three seconds under normal conditions.
- **NFR-04 (Maintainability)**: The backend and frontend must be clearly separated and independently deployable. The project structure should follow established conventions for its respective framework.
- **NFR-05 (Deployability)**: The frontend and backend must be clearly separated and independently deployable via automated CI/CD pipelines to modern cloud platforms. Local development should still be achievable with a single Docker Compose command.

2.6 Architecture Analysis

This section justifies the major technology choices by comparing the available alternatives against the requirements established in the preceding sections. The selected stack was chosen for its fit with the required functionality, its maturity, and its suitability for a bachelor thesis prototype.

2.6.1 Frontend Framework

The core capabilities of the system, such as course administration, content cloning, contextual messaging, and AI assistance, require a responsive and component-rich user interface that can manage shared state across a course-material viewer, a chat panel, and an AI assistant panel rendered simultaneously on the same screen.

React [1] was selected as the frontend foundation because its component model maps naturally onto the StudySpace interface, where the main screen can be decomposed into independent panels with clearly separated responsibilities. Its declarative approach and unidirectional data flow also make state changes easier to reason about in a prototype with multiple interacting views.

Vue.js [2] and Angular [3] were considered as alternatives. Vue offers a gentler learning curve and a lighter framework model, while Angular provides

a more opinionated and feature-complete environment. For StudySpace, however, React was the better fit because the project benefits more from its ecosystem breadth, mature TypeScript support [4], and the availability of reusable UI primitives for complex layouts.

React is used through the Next.js [5] meta-framework, which provides file-based routing, server-side rendering where applicable, and built-in optimisation features. This reduces boilerplate and keeps the project structure explicit through the directory layout. Styling is handled by Tailwind CSS [6] in combination with shadcn/ui [7], which provides accessible component primitives that can be adapted with utility classes.

State management across the application uses React's Context API [8] and custom hooks. This is sufficient for the project scope and provides a lightweight and predictable data flow without introducing an external global state library.

2.6.2 Backend Framework

The system requires robust access control with role-based authorisation. Furthermore, the content extension workflows and AI assistance feature involve complex transactional operations over relational data. These requirements favour a statically typed backend with mature transaction management and security libraries.

Java 21 with Spring Boot 3 [9, 10] provides a statically typed, object-oriented foundation with a mature ecosystem. Spring Boot's convention-over-configuration approach applies sensible defaults automatically, embeds a Tomcat web server, and offers first-class integration with Spring Security [11], Spring Data JPA [12], Spring WebSocket [13], and Springdoc OpenAPI [14], which are all relevant for StudySpace. Virtual Threads, introduced in Java 21 [15], improve the throughput of concurrent request handling without requiring reactive programming.

Node.js [16] with NestJS [17] or Express [18] excels in I/O-bound tasks due to its event-driven architecture [19]. However, its single-threaded event loop is less attractive for the multi-step transactional operations required by the content extension system, and the ORM ecosystem does not reach the integration depth and stability of Spring Data JPA.

Python [20] with Django [21] or FastAPI [22] is excellent for rapid prototyping and AI integration. Python's dynamic typing, however, makes long-term maintenance of larger backend codebases more demanding without strict tooling, and the Global Interpreter Lock can become a bottleneck for highly concurrent CPU-bound tasks [23].

Java 21 with Spring Boot 3 was selected because the controller-service-repository layered model [24, 25] maps directly onto the StudySpace domain structure, Spring Security provides JWT-based authentication and method-level authorisation, and Spring's transaction management supports referential integrity across clone and approval workflows.

2.6.3 Database and Vector Store

The application's data model is inherently relational: users own workspaces, workspaces belong to courses, clone records track inheritance, and proposals reference both student materials and instructor courses. The content extension system therefore requires ACID transaction guarantees across multi-entity writes.

PostgreSQL [26] is a mature, open-source relational database management system that follows the SQL standard and supports complex queries, transactions, and foreign-key constraints. It integrates cleanly with the Java backend through JDBC, and higher-level access is provided by Spring Data JPA [12] with Hibernate [27] as the ORM provider.

MongoDB [28] is optimised for unstructured or semi-structured data and horizontal scalability. However, it is less suitable for a domain that depends on relational consistency, schema enforcement, and atomic updates across multiple related entities. Using a document store would require significant denormalisation and would complicate consistency when a Course Material is approved and must be reflected in multiple places at once.

PostgreSQL was selected because the StudySpace data model is fundamentally relational, ACID transactions are required for the clone and approval workflows, and foreign-key constraints enforce referential integrity between proposals and their source materials. Additionally, the AI assistant leverages the pgvector extension [29] to provide similarity search over document embeddings directly within the same database, eliminating the need for a separate vector store.

2.6.4 Authentication and Authorisation

The access control system requires stateless authentication suitable for a REST API [30] consumed by a single-page application, with role-based restrictions enforced on the server side.

JSON Web Tokens (JWT) [31] with Spring Security [11] enable stateless authentication: after login, the server issues a signed token that the client stores and sends with every subsequent request. The server validates the token signature without consulting a session store, which simplifies horizontal scaling.

Server-side sessions with Spring Session [32] require a shared session store, such as Redis, across instances and add network latency to every request. For a stateless REST API served to a browser-based SPA, that overhead is unnecessary.

OAuth 2.0 only [33] would prevent users without a Google or GitHub account from registering, which is inappropriate for a faculty application where students may not use third-party providers for academic accounts.

JWT with Spring Security was selected because it achieves stateless session management, requires no external session store, and fits naturally into Spring

Security's filter chain. Google OAuth 2.0 login is supported as an optional alternative authentication path alongside the primary local credential flow.

2.6.5 Real-Time Communication

The Contextual Messaging System requires peer-to-peer discussion to occur in real time. This necessitates bidirectional event delivery to all connected participants, ruling out traditional request-response and simple polling-based approaches.

WebSocket [34] with STOMP [35] over SockJS [36] layers a publish-subscribe model on top of the raw WebSocket connection. Spring Boot registers a broker endpoint and routes messages to per-session topics, allowing the server to push participant-list updates and activity events to all subscribers without the clients polling.

Server-Sent Events (SSE) [37] provide a unidirectional push channel from server to client. They therefore do not support the bidirectional interaction model required when clients must also send activity events back to the server.

Long polling [38] can approximate real-time delivery but requires each client to maintain a pending request and re-establish it after every response, producing significant per-client overhead on the server.

WebSocket with STOMP over SockJS was selected because the bidirectional subscription model covers both server-to-client pushes and client-to-server messages over a single connection, and Spring Boot's built-in STOMP message broker requires no additional infrastructure.

2.6.6 Quality Assurance

Ensuring the reliability of the StudySpace web application requires a robust quality assurance strategy, encompassing both isolated component testing and full-system end-to-end verification. The selection of appropriate testing frameworks must account for the project's reliance on Next.js, the need for rapid developer feedback, and the complexity of real-time browser interactions.

2.6.6.1 Unit and Integration Testing

Unit and integration tests verify individual application components, such as the contextual chat panel or the course material upload logic, in isolation from the full backend server. The two primary candidates for the React ecosystem are Jest and Vitest.

Jest [39] is a mature, widely adopted testing framework maintained by the open-source community. It provides a comprehensive set of assertion libraries and mocking utilities out of the box. However, Jest relies on a traditional Node.js execution model and typically requires more configuration to handle modern ECMAScript modules and TypeScript smoothly.

Vitest [40] is a newer framework built on top of Vite. It is designed to be API-compatible with Jest, allowing developers to use familiar syntax for assertions and mocks, but it uses a more modern build pipeline for module resolution and transformation. This results in faster test execution and a shorter feedback loop during development. In the context of StudySpace, where frequent iterations on the user interface and state management hooks are required, minimising feedback time is important.

Given the performance advantages of its execution model and its compatibility with the React Testing Library [41], Vitest was selected as the unit and integration testing framework.

2.6.6.2 End-to-End Testing

End-to-end testing simulates real user behaviour by driving an actual web browser, ensuring that the frontend, backend, and database interact correctly. This is particularly important for StudySpace, as features like the Merge Proposal workflow and the real-time Contextual Messaging System involve complex cross-tier interactions. Cypress and Playwright are the leading solutions for modern web applications.

Cypress [42] offers a highly visual, interactive test runner that executes directly within the browser alongside the application under test. This architecture provides excellent debugging capabilities and a shallow learning curve. However, Cypress traditionally struggles with testing multi-tab workflows and cross-origin navigations, which can be problematic when testing OAuth flows or interactions that open course materials in new tabs.

Playwright [43], developed by Microsoft, takes a different architectural approach by communicating with browsers over the Chrome DevTools Protocol. It natively supports parallel test execution across multiple browser engines and handles multi-tab, multi-origin scenarios more effectively. Playwright also includes a trace viewer that captures DOM snapshots, network requests, and console logs for failed tests, which simplifies debugging.

For StudySpace, the ability to test complex user journeys efficiently is paramount. The native parallel execution model of Playwright [44] reduces the time required to run the full E2E suite, offering a performance advantage over standard Cypress setups. Additionally, its robust handling of modern web APIs and multi-context scenarios ensures that the real-time communication channels and file viewing features can be tested reliably. Consequently, Playwright was chosen as the end-to-end testing framework for the project.

2.7 Conclusions from Analysis

The analysis of the problem space, requirements, and existing platforms reveals a clear need for a unified system. None of the evaluated solutions successfully integrate structured course administration (FR-01) with a student-

driven content extension system (FR-02) and contextual peer messaging (FR-03). Furthermore, the necessity for an AI assistant grounded directly in the course materials (FR-04) cannot be met by general-purpose collaborative tools. These identified gaps and defined requirements establish the boundaries and constraints that directly drive the architectural and technical design decisions presented in the following chapter.

Chapter 3

Design

Based on the requirements and gaps identified in Chapter 2, the following design decisions were made.

This chapter translates the requirements and architecture analysis established in Chapter 2 into a concrete technical design. The domain model defines the entities and their relationships. The architectural design outlines the overarching application structure, followed by specific design decisions for file storage and the integration of large language models (LLM). The database schema documents the persistent data model. Finally, the API design and user interface design detail how the system will be accessed and experienced by its users.

3.1 Domain Model

The domain model consists of the entities that carry the application's persistent state. To ensure clarity, the complete model is divided into four functional subsystems corresponding to the core features described in Chapter 2. Each subsystem and its entities are described below. The domain model diagrams in this section were produced using DrawDB [**DrawDB**], a browser-based entity-relationship diagram editor. Additionally, a bird's-eye view of all entity relationships is presented in the complete domain model diagram in Appendix A, and the schema is provided in the enclosed electronic media as an attached resource.

3.1.1 User Management and Access Control

User is the central actor entity. It stores the account credentials (username, email, and bcrypt-hashed password), the display name, a profile picture URL, and a role field that takes one of two values: `STUDENT` or `INSTRUCTOR`. The entity also includes fields for a cumulative study-time counter and a current-

streak counter to support future gamification features. A user can create many `Course` records (as an Instructor), own many `StudentWorkspace` records (as a Student), and participate in many `StudySession` records through the `SessionParticipant` junction entity.

Activity is designed to represent study-related events, carrying a type, an optional text message, a timestamp, and a reference to the owning user, optionally linked to a specific study session.

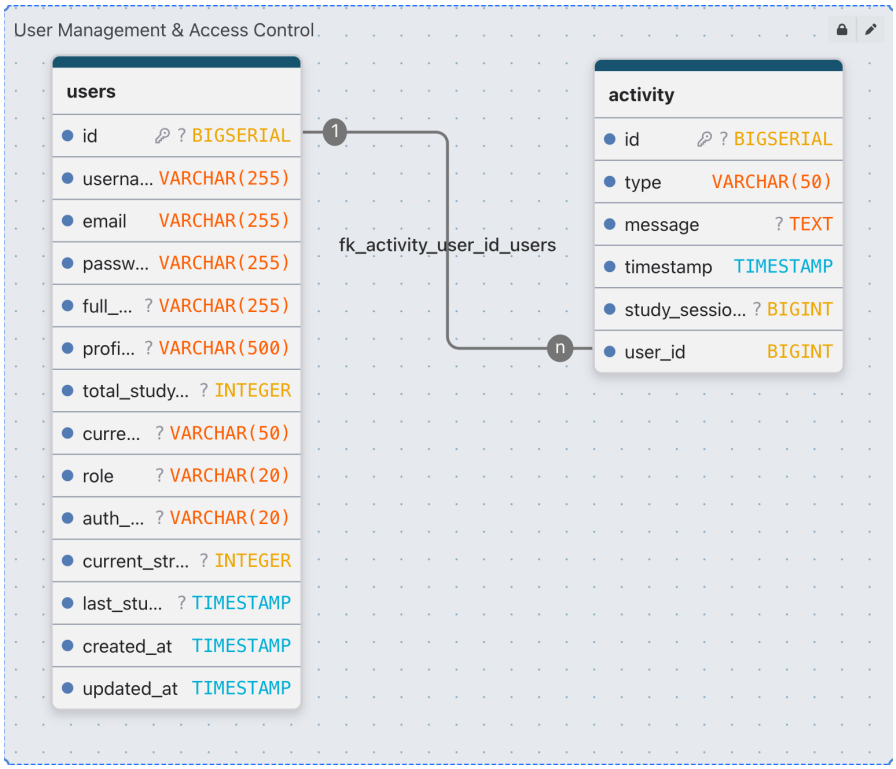

Figure 3.1 User management and access control domain model

3.1.2 Course Administration Module

Course represents a published course created by one Instructor. It carries a title, a text description, a boolean `isPublished` flag that controls student visibility, and a many-to-one relationship to its owning `User` (the Instructor). Deleting a `Course` cascades to all its `CourseSection` and `CourseEnrollment` records via `CascadeType.ALL` with orphan removal, ensuring no orphaned sections or enrollments remain after a course is removed.

CourseSection organises Course Materials within a course into named, ordered groups. It carries a title, an optional text description, and an `orderIndex` integer that determines the display sequence. Each section belongs to

exactly one `Course`, and each section owns an ordered list of `CourseMaterial` records. Deleting a section cascades to all its materials.

CourseMaterial represents a single file uploaded into a course section. It stores a display title, the file storage URL, the inferred file type as a `MaterialType` enum (`PDF`, `SLIDES`, `VIDEO`, `IMAGE`, `OTHER`), the original filename, an optional `contributorName` field (populated when the material originated from an approved Merge Proposal), and a timestamp. Each `CourseMaterial` belongs to exactly one `CourseSection`.

CourseEnrollment is the join entity that records a student's membership in a course. It carries an `EnrollmentStatus` field (`ACTIVE` or `DROPPED`) and the timestamp of the initial enrollment. A unique constraint on the pair `(course_id, student_id)` prevents duplicate enrollment rows. Setting the status to `DROPPED` rather than deleting the row preserves the enrollment history and allows re-activation.

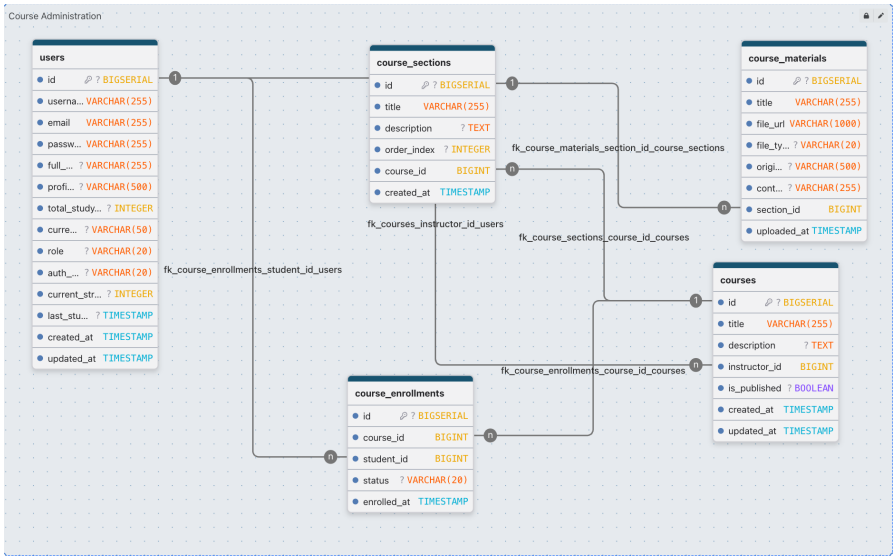

Figure 3.2 Course administration domain model

3.1.3 Content Extension System

StudentWorkspace is the root entity of a student's Private Workspace. It holds a name, a description, a reference to the owning `User`, and a list of `WorkspaceSpace` records. The workspace is the container that the student manages independently; it does not belong to any course and cannot be seen or modified by the Instructor.

WorkspaceSpace represents a single cloned or manually created space within a Private Workspace. It carries a title, a description, an optional foreign key to the `Course` it was forked from (`forkedFromCourse`), and a boolean

`isPublished` flag used when a space is surfaced in a public gallery. Each space owns an ordered list of `WorkspaceSection` records.

WorkspaceMaterial mirrors `CourseMaterial` within the student's space. Two flags encode the copy-on-write behaviour: `isReference = true` means the material points to the original course file URL and no physical copy was made; `isHidden = true` implements a soft-delete that hides the row from the student's view without physically removing it, which is necessary because a contribution proposal may hold a foreign-key reference to the material row and deleting it would violate referential integrity.

ContributionProposal links a student's `WorkspaceMaterial` to a target `Course` and optionally a target `CourseSection`. Its `ProposalStatus` enum field (`PENDING`, `APPROVED`, `REJECTED`) drives the Instructor review workflow. When no existing section is targeted, the `proposedSectionTitle` field holds the name of a new section to be created on approval. The `reviewedAt` timestamp is set when the Instructor takes action, and the `contributorDisplayName` is copied from the student's profile at submission time so it remains stable even if the student later changes their name.

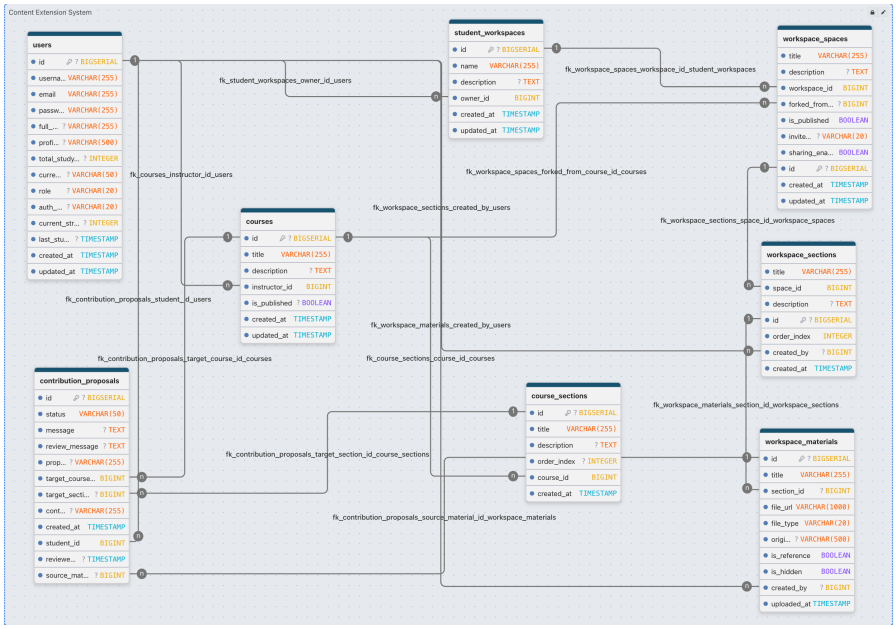

Figure 3.3 Content extension domain model

3.1.4 Contextual Messaging & Collaboration

StudySession and **SessionParticipant** manage real-time group study rooms, tracking the session lifecycle, visibility, and the cumulative study time of each participating student.

SpaceGuest and **SpaceMessage** enable real-time collaboration within a published Workspace Space, linking external users and their chat messages to the target space.

Conversation persists the AI chat memory for a single user session. Rather than storing every message individually in a single blob, the entity is part of a normalised schema tracking the user owner and the conversation title. Individual message turns are stored in the **Message** table. The conversation's primary key is a client-generated UUID string, so no database sequence is required.

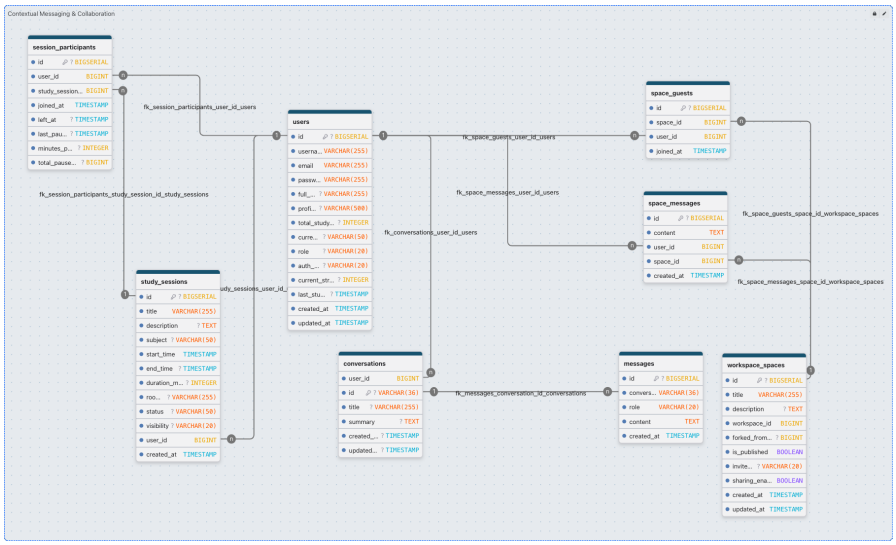

Figure 3.4 Contextual messaging and collaboration domain model

3.2 Architectural Design

At the highest level, StudySpace is a web application that separates the browser-based user interface from the server-side application logic and the persistent data store. The frontend communicates with the backend over a standard request-response channel for RESTful operations and a persistent real-time channel for live events such as chat messages. The backend handles all application logic, access control, and external API integrations, while the data store manages all persistent state. These tiers are orchestrated as isolated containers, ensuring a consistent and portable deployment environment.

3.2.1 System Architecture

StudySpace follows a three-tier architecture. The *presentation layer* is a Next.js and React application running in the browser; it communicates with

the backend via the REST API for most operations and via a persistent WebSocket (STOMP/SockJS) connection during active study sessions. The *business logic layer* is a Spring Boot application that handles authentication, data access, real-time session broadcasting, and LLM API calls. The *data layer* is a PostgreSQL database that stores all persistent data.

For local development, all three tiers can be orchestrated using `docker-compose.yml` [45] to run in the same virtual network. For production deployment, the architecture is decoupled: the frontend is deployed to a dedicated hosting platform (e.g., Vercel [46]), the backend runs as a containerised service on a cloud platform (e.g., Render [47]), and file storage is handled by an S3-compatible service (e.g., Tigris [48]). This decoupling ensures independent scalability and simplifies deployment pipelines.

3.2.2 Logical Architecture

The server-side application follows the Model-View-Controller (MVC) layered pattern [24, 25]. The domain model diagrams in this chapter were generated using PlantUML, a plain-text markup tool for UML diagram generation. The *controller layer* receives HTTP requests, delegates to the appropriate service, and serialises the response. The *service layer* contains all business logic, orchestrates data access, and enforces domain rules such as ownership checks and status transitions. The *repository layer* provides data access through named query methods and custom queries for complex lookups. Entity classes map the relational schema to Java objects. Data Transfer Objects (DTOs) define the API contract and decouple the internal entity structure from the shape of the JSON responses, so the two can evolve independently.

The backend is organised into the following packages, each with a single responsibility and dependencies flowing only from controller to service to repository:

The frontend follows an equivalent layered approach, separating presentation (UI components and pages) from business logic (hooks and contexts) and data access utilities (types and API integrations).

Table 3.1 Backend package structure

Package	Responsibility
`controller`	Defines REST endpoints, maps HTTP requests to service calls, and validates incoming request bodies.
`service`	Contains all business logic. Services are called by controllers and may call one or more repositories. Transactions are managed at this layer.
`repository`	Spring Data JPA repositories, providing CRUD operations and custom JPQL queries for complex data access.
`entity`	JPA entity classes mapped to database tables.
`dto`	Data Transfer Objects used as the API contract, decoupling the API response shape from the internal entity structure.
`mapper`	Stateless mapper classes that convert between entity and DTO representations.
`security`	JWT filter, `UserDetailsService` implementation, and Spring Security configuration.
`config`	Application-level configuration beans: CORS policy, Swagger setup, and WebSocket registration.
`exception`	Custom exception classes and a global `@ControllerAdvice` handler that maps them to appropriate HTTP error responses.
`types`	Shared enumerations used across multiple layers, such as `UserRole`, `ProposalStatus`, and `MaterialType`.
`util`	Stateless utility helpers, such as date and time formatting utilities.

3.3 File Storage

FR-01 and FR-02 require file upload and retrieval for Course Materials and Private Workspace materials. The storage backend must be swappable between a local development mode and a production cloud target.

Local file storage (writing files to a directory under the working directory) is zero-cost and requires no external service, making it highly efficient for local development. However, it is unsuitable for production because files are lost when a PaaS container is recreated, and the storage cannot be shared across multiple server instances for horizontal scaling.

Distributed Object Storage (Tigris [48]) provides high durability, direct URL serving, and persistent scaling. Tigris offers an Amazon S3-compatible API [49], allowing the use of the standard AWS SDK v2. This guarantees that uploaded files persist safely independent of the application container's lifecycle.

A `FileStorageService` interface with two concrete implementations was designed: `LocalFileStorageService` for development and `S3FileStorageService` for production. The active implementation is selected automatically by the Spring `docker` profile, keeping the storage backend transparent to the rest of the application and satisfying NFR-04. The interface defines three methods: `store`, `copy`, and `delete`, which are the only storage operations needed by FR-01 and FR-02.

3.4 LLM API Integration

FR-04 requires an LLM API capable of answering questions grounded in a provided document context within a single prompt, without fine-tuning.

Google Gemini API (gemini-3-flash-preview) provides a large context window, competitive quality on open-domain question answering, and an official Java SDK that integrates directly with the Spring Boot application. The free tier is sufficient for prototype-level usage.

OpenAI GPT-5.4-mini exposes a compatible REST interface and achieves comparable quality on question answering tasks. However, the Java SDK is less mature than the official Google GenAI SDK, and the cost per token is higher for the same context window size at the time of writing.

The Google Gemini API was selected as the default provider because the official Google GenAI Java SDK [50] integrates directly with Spring Boot without additional HTTP client configuration, the `gemini-3-flash-preview` model's context window comfortably accommodates the semantically relevant chunks retrieved from the `pgvector` database required by FR-04, and the free tier capacity is sufficient for the prototype.

3.5 Database Schema

The relational schema follows directly from the domain model described in section 3.1. Each entity maps to a dedicated table, and all foreign-key relationships are enforced at the database level.

The primary tables correspond to the core domain entities: users, courses, course enrollments, course sections and materials, and their workspace counterparts. The `course_enrollments` table enforces a unique constraint on the column pair `(course_id, student_id)` to prevent duplicate enrollment rows. The `workspace_materials` table carries the two boolean flags `isReference` and `isHidden` that implement the copy-on-write and soft-delete semantics described in section 3.1.

The AI conversation memory utilises two tables: `conversations` stores the overarching context and the rolling `summary` of compressed long-term memory, while the `messages` table records individual chat interactions. This separation allows efficient querying of recent conversation history. For document-grounded question answering (FR-04), the system uses the `document_chunks` table. This table leverages the `pgvector` PostgreSQL extension to store high-dimensional embeddings of document segments, enabling fast semantic similarity search during the retrieval-augmented generation process.

3.6 API Design

The backend exposes a RESTful API following standard HTTP conventions. Resources are identified by URL paths, and operations are expressed through HTTP methods (`GET`, `POST`, `PUT`, `DELETE`). All protected endpoints require a valid JWT token in the `Authorization: Bearer <token>` header.

Authentication follows a four-step flow. The client sends credentials to `POST /api/auth/login`. The server validates the credentials and returns a signed JWT token. The client stores the token in memory and includes it in the `Authorization` header of all subsequent requests. The server's `JwtAuthenticationFilter` validates the token on each request before passing control to the controller.

The full API is documented through the Swagger UI [51] available at `/swagger-ui/index.html` when the backend is running, auto-generated by Springdoc OpenAPI from controller annotations.

3.7 User Interface Design and Wireframes

The design process began by producing low-fidelity wireframes to establish the basic structural layouts and core user flows before initiating frontend development. These initial wireframes represent rough draft layouts, using simple crossed boxes to outline visual element placeholders and minimal outlines to de-

fine interface boundaries. The design focused on the most critical interactions of the application: querying the AI assistant (shown in Figure 3.5), submitting a merge proposal (illustrated in Figure 3.6), and reviewing contribution proposals (depicted in Figure 3.7). The wireframes were designed in Figma, a browser-based collaborative interface design tool.

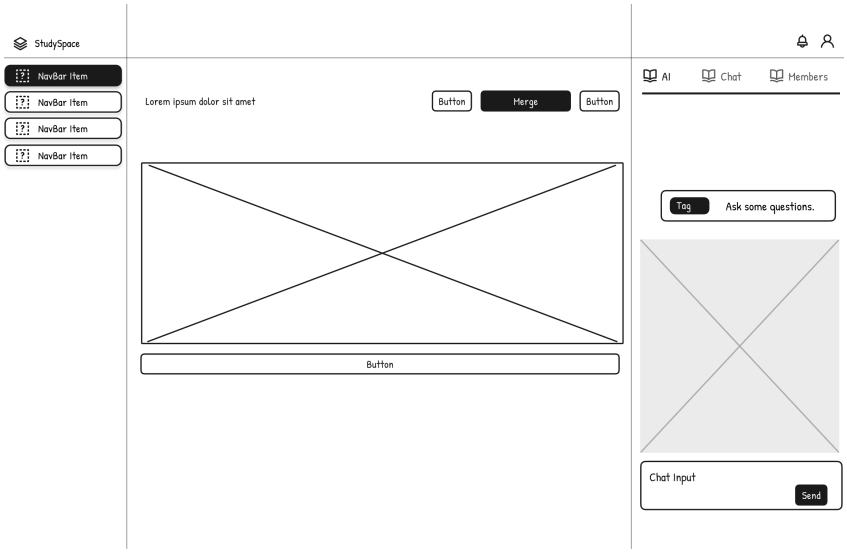

Figure 3.5 Low-fidelity wireframe: AI assistant conversational interface

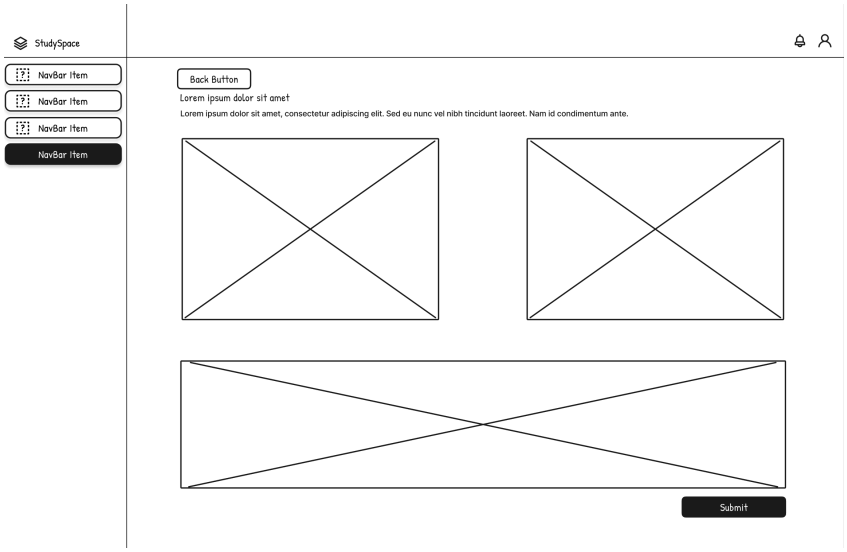

Figure 3.6 Low-fidelity wireframe: Student merge proposal submission

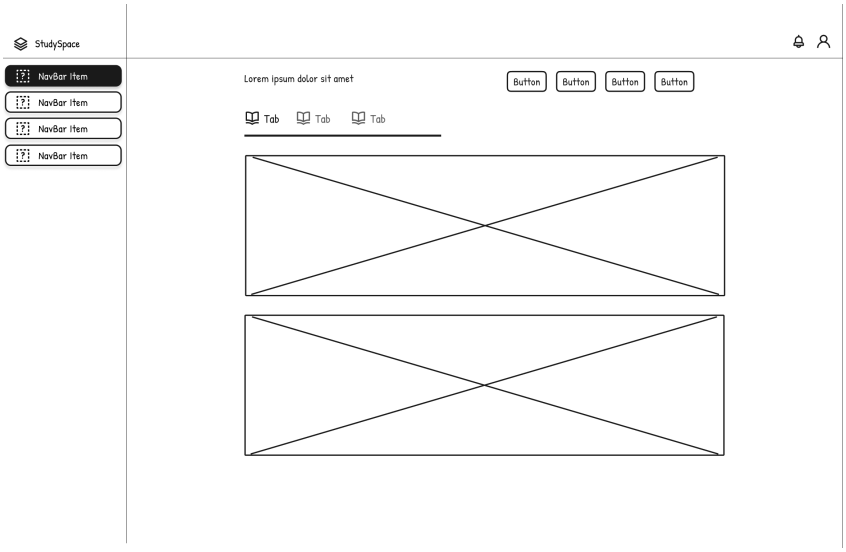

Figure 3.7 Low-fidelity wireframe: Instructor contribution review interface

Chapter 4

Implementation

This chapter documents how the five core features defined in Chapter 2 were translated into working code. Each feature section follows the same structure: the purpose and technical challenge are contextualised first, the application screenshot shows the feature in action, the underlying logic is explained in prose, and a targeted code listing illustrates the most important implementation decision. Section 4.6 covers automated and usability testing, and section 4.8 describes the project organisation.

4.1 Course Administration Module (FR-01)

The course administration feature addresses FR-01: Instructors must be able to create and manage courses, upload Course Materials into named sections, and control student enrollment. The technical challenge in this feature is keeping the file storage backend transparent to the business logic so that the development environment (local disk) and the production environment (Amazon S3-compatible cloud storage [49]) can be swapped by changing a Spring profile, without any changes to the service layer.

The course management interface is shown in Figure 4.1. The page presents the Instructor with a list of sections and a file upload button per section. Selecting a file triggers a multipart form data request to the backend; the Instructor does not see storage paths or file types, which are resolved automatically by the backend.

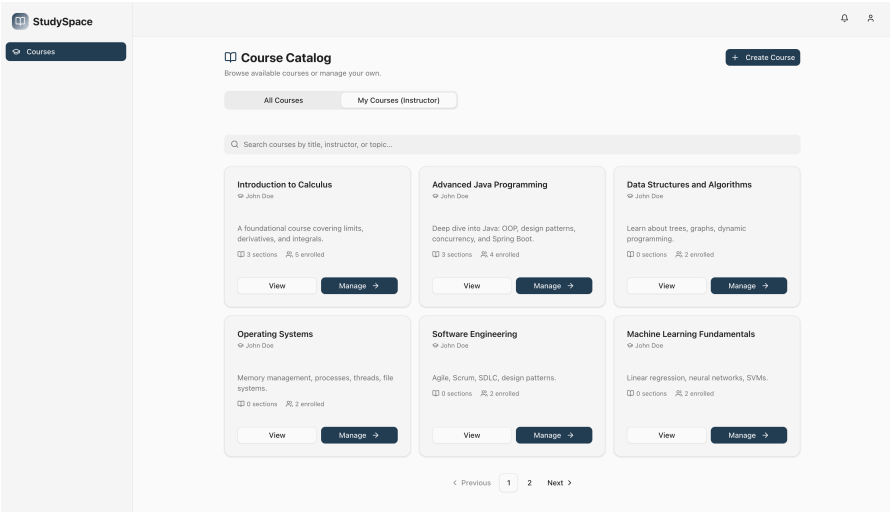

Figure 4.1 Instructor course management page: section list with upload controls

The `CourseService` delegates all file operations to the `FileStorageService` interface. When `uploadMaterial` is called, the service stores the file, infers the `MaterialType` from the file extension, and persists the resulting URL in the `CourseMaterial` entity. The ownership check in `assertInstructor` ensures that only the course owner can modify sections or materials; any other authenticated user receives an HTTP 403 response. The listing below shows the upload method, which encapsulates this flow in under twenty lines.

```
public CourseMaterialDTO uploadMaterial(Long sectionId, Long
    requestingUserId,
                                        MultipartFile file, String
    title) {
    CourseSection section = findSection(sectionId);
    // Only the course instructor may upload
    assertInstructor(section.getCourse(), requestingUserId);

    // FileStorageService is injected; impl selected by Spring
    profile
    String fileUrl = fileStorageService.store(file, "courses");
    MaterialType fileType = detectFileType(file.getOriginalFilename()
    );

    CourseMaterial material = CourseMaterial.builder()
            .title(title).fileUrl(fileUrl).fileType(fileType)
            .originalFileName(file.getOriginalFilename())
            .section(section).build();
    return toMaterialDTO(materialRepository.save(material));
}
```

Code listing 4.1 Code listing 4.1 — Course material upload with storage abstraction (CourseService.java)

The `FileStorageService` interface defines three methods (`store`, `copy`, `delete`), and the active implementation is injected by Spring. This design satisfies NFR-04 (maintainability) by keeping the service layer unaware of the storage backend, and means that switching to a cloud provider for production requires only an environment variable change, not a code change.

4.2 Content Extension System (FR-02)

The Content Extension System addresses FR-02: students must be able to clone a course into a Private Workspace, modify it freely, and submit a Merge Proposal for the Instructor to review. The key technical challenges are the copy-on-write cloning strategy (avoiding redundant file copies for shared base materials) and the referential-integrity constraint that prevents hard deletion of materials that are still referenced by pending proposals.

The student Private Workspace interface and its Merge Proposal submission dialog are shown in Figure 4.2. Once the student submits a proposal, the Instructor can review it through the interface illustrated in Figure 4.3. The workspace view shows the student a mirror of the course structure. An upload button per section allows the student to add new materials. The Merge Proposal dialog appears when the student selects one or more materials and clicks the contribute button; the student picks a target section (or names a new one) and submits.

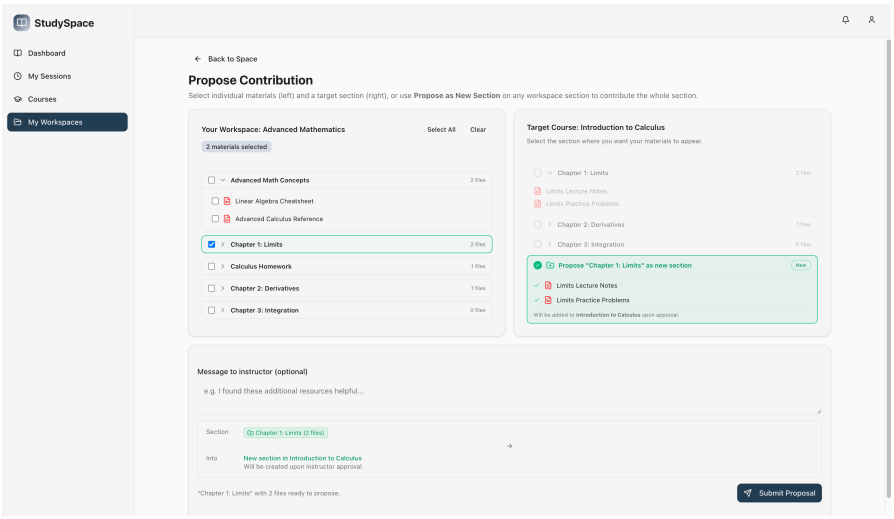

Figure 4.2 Student Merge Proposal View: Submission dialog from the Private Workspace

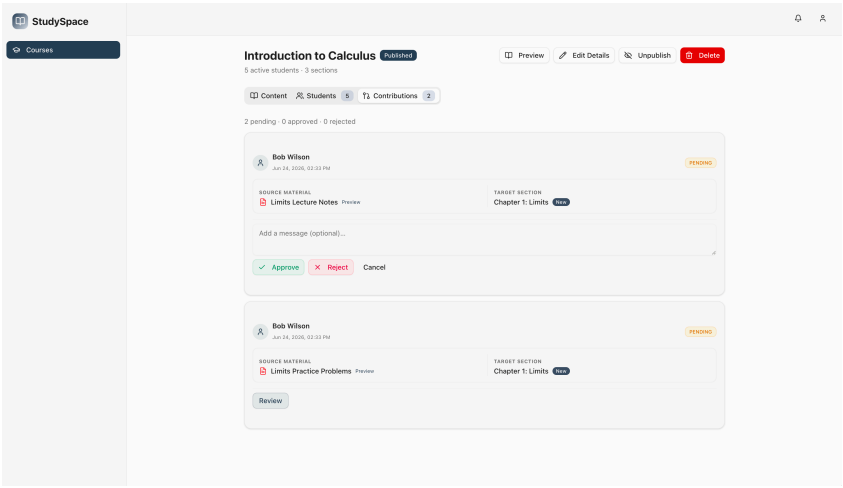

Figure 4.3 Instructor Review View: Evaluating a student's Merge Proposal

When `forkCourse` is called on `WorkspaceService`, the backend iterates over all sections and Course Materials of the source course and creates `WorkspaceSection` and `WorkspaceMaterial` rows that point to the original file URLs (`isReference = true`) rather than copying the files. Only materials uploaded by the student later are stored as new files (`isReference = false`). When a student removes a reference material from their view, `deleteMaterial` in `WorkspaceService` checks for active `ContributionProposal`s. If a proposal exists, it sets `isHidden = true` on the row instead of deleting it to preserve referential integrity. Furthermore, to prevent the deletion of a physical file that is still in use elsewhere, the system employs a reference-counting mecha-

nism. Before a physical file is removed from storage, the backend verifies that no other `CourseMaterial` or `WorkspaceMaterial` points to the same `fileUrl`. This ensures that an instructor deleting an original course file will not break the cloned references in student workspaces, and vice versa. The listing below shows this logic.

```
public void deleteMaterial(Long materialId, Long userId) {
    WorkspaceMaterial material = materialRepository.findById(
    materialId)
            .orElseThrow(() -> new RuntimeException("Material not
    found: " + materialId));
    // Authorisation checks omitted for brevity

    boolean hasProposal = proposalRepository.existsBySourceMaterialId
    (materialId);
    if (hasProposal) {
        // Soft-delete: keep the row so ContributionProposal FK stays
     intact
        material.setIsHidden(true);
        materialRepository.save(material);
    } else {
        if (!Boolean.TRUE.equals(material.getIsReference())) {
            String fileUrl = material.getFileUrl();
            materialRepository.delete(material);

            // Reference Counting: Delete file only if no other
    entity uses it
            boolean hasCourseRefs = courseMaterialRepository.
    existsByFileUrl(fileUrl);
            boolean hasWorkspaceRefs = materialRepository.
    existsByFileUrl(fileUrl);
            if (!hasCourseRefs && !hasWorkspaceRefs) {
                fileStorageService.delete(fileUrl);
            }
        } else {
            materialRepository.delete(material);
        }
    }
}
```

Code listing 4.2 Code listing 4.2 — Delete strategy with reference counting (WorkspaceService.java)

The two-branch delete strategy preserves referential integrity without requiring a separate audit table or a deferred constraint. The `WorkspaceSectionDTO` mapper already filters hidden materials before returning the section content to the frontend, so the student's view is consistent with the logical deletion even though the database row remains.

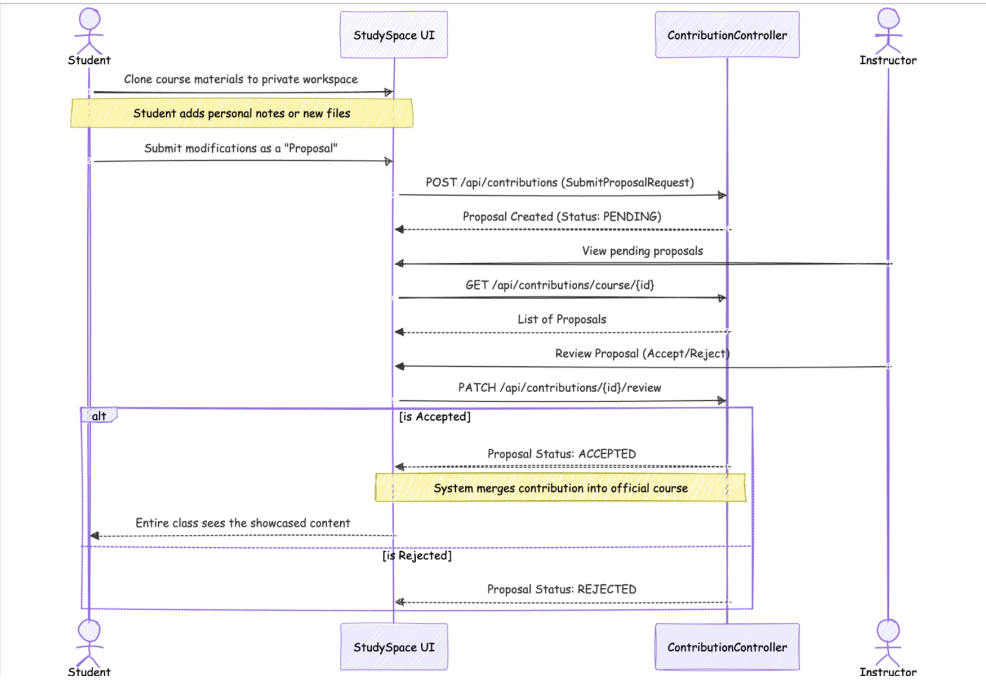

Figure 4.4 Merge proposal lifecycle sequence diagram: From student submission to instructor review and acceptance

4.3 Contextual Messaging System (FR-03)

The Contextual Messaging System addresses FR-03: users must be able to post messages in the workspace chat and attach a Contextual Anchor to a specific Course Material so that other users can open the material from within the message. The technical challenge is embedding a non-editable interactive chip into a plain-text-like input field so that the anchor is preserved through submission, serialisation, and re-render.

The workspace viewer and its integrated contextual chat panel are shown in Figure 4.5. The chat panel is shown after a student typed `@lecture` and the autocomplete dropdown appeared. After selecting a Course Material, a styled chip replaces the `@`-query in the input. The submitted message is rendered with a clickable badge that opens the file.

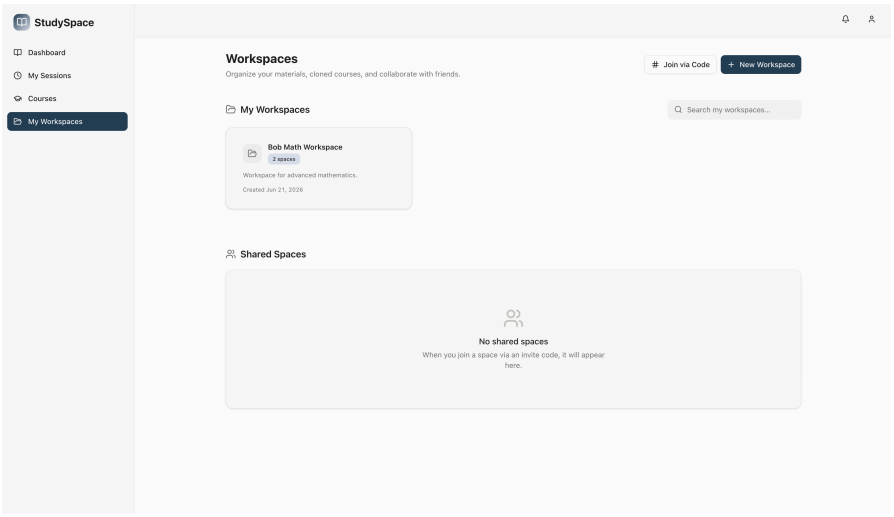

Figure 4.5 Workspace viewer: Course Material list with contextual chat panel

The message input uses a `<div contentEditable>` element rather than a `<textarea>` because `<textarea>` can only hold plain text, whereas a `contentEditable` div can host arbitrary DOM nodes [52]. When the user selects a Course Material from the autocomplete, the `insertTag` function deletes the `@<query>` text and inserts a `<span>` element with `contentEditable="false"` and `data-materialId`, `data-materialTitle`, and `data-materialUrl` attributes. Before submission, `parseInput` serialises the span into the token `@[<id>:<title>]`, which is stored as the message text. On render, a regular expression parses the token back into a `<Badge>` component. The listing below shows the serialisation step.

```
// Walks the contentEditable div's child nodes and builds the token string
const parseInput = (): string => {
  if (!inputRef.current) return "";
  let question = "";
  for (const node of Array.from(inputRef.current.childNodes)) {
    if (node.nodeType === Node.TEXT_NODE) {
      question += node.textContent;
    } else if (node.nodeType === Node.ELEMENT_NODE) {
      const el = node as HTMLElement;
      if (el.dataset.materialId) {
        // Non-editable chip -> portable anchor token
        question += `@[${el.dataset.materialId}:${el.dataset.materialTitle}]`;
      } else {
        question += el.textContent ?? "";
      }
    }
  }
```

```
  return question.trim();
};
```

Code listing 4.3 Code listing 4.3 — DOM serialisation to portable anchor token (member-chat.tsx)

The token format `@[id:title]` is compact, survives JSON serialisation, and is unambiguous to parse with a single regular expression on the render side. Storing the material ID inside the token allows the backend to validate or re-fetch the material metadata if needed, while the title provides a human-readable fallback if the material is later renamed.

4.3.1 Workspace Collaboration

To facilitate peer-to-peer discussion, Private Workspaces must support collaboration among multiple students. The system achieves this by allowing the workspace owner to add other users as members. The `WorkspaceService` manages this by persisting a list of `SpaceMemberDTO` objects associated with the space. Before a user can post or view messages in the chat, the `assertMember` security check verifies their membership. This transforms the Private Workspace from an isolated clone into a collaborative study environment, directly supporting the interactive requirements of FR-03.

4.3.2 Real-Time Communication via WebSockets

For the discussion to be truly interactive, messages must be delivered to all workspace members instantly without client-side polling. This is implemented using WebSockets with STOMP over SockJS (as justified in section 2.6.5). While earlier iterations used WebSockets exclusively for study sessions, the infrastructure was extended to power the Contextual Messaging System.

When a user submits a message, the `SpaceChatController` intercepts the HTTP `POST` request, saves the message to the database, and immediately broadcasts the message payload to the STOMP destination corresponding to the workspace ID.

```
@PostMapping
public ResponseEntity<SpaceMessageDTO> sendMessage(
        @PathVariable Long spaceId,
        @RequestBody Map<String, Object> payload) {

    // Parse payload and save SpaceMessage entity...
    SpaceMessageDTO dto = SpaceMessageDTO.builder()
            .content(message.getContent())
            .spaceId(message.getSpace().getId())
            // ... other fields omitted for brevity
            .build();

```

```
13      // Broadcast to space members over WebSocket channel
14      String destination = "/topic/space/" + spaceId + "/chat";
15      messagingTemplate.convertAndSend(destination, dto);
16
17      return ResponseEntity.status(HttpStatus.CREATED).body(dto);
18  }
```

Code listing 4.4 Code listing 4.4 — WebSocket broadcast for workspace chat (SpaceChatController.java)

On the frontend, the React application uses a WebSocket hook to subscribe to the `/topic/space/{spaceId}/chat` channel. Upon receiving a message event, the client appends the new message to the local component state, triggering an immediate UI update for all connected members.

4.4 AI-Based Content Querying (FR-04)

The AI querying feature addresses FR-04: users must be able to ask natural-language questions about the content of the currently open workspace and receive answers grounded in that content. The technical challenges are normalising three distinct document URL formats into a single byte stream, constructing a structured prompt that keeps the model grounded in the document, and handling API rate limits gracefully so the student always receives a useful response.

The AI assistant panel is shown in Figure 4.6. The panel shows the student's question at the top and the LLM-generated answer below, displayed with a typewriter animation. The panel is collapsible so the student can read the Course Material alongside the answer.

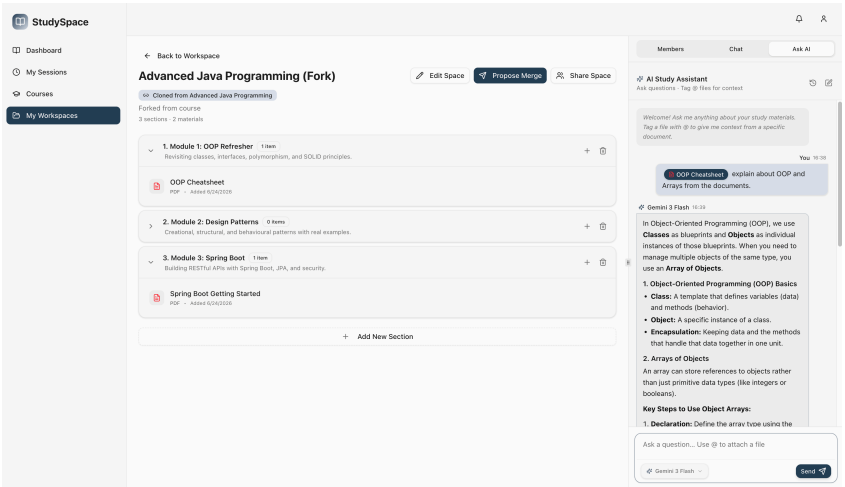

Figure 4.6 AI assistant panel: student question with grounded answer and typing animation

When the user submits a question, the frontend sends the question and the selected workspace references to `POST /api/chat/query`. The `DocumentVectorService` retrieves semantically relevant document excerpts from the `pgvector` database by comparing the vector embedding of the user's question against the stored document chunks (which are extracted via Apache PDFBox [53] and embedded during file upload). The `PromptBuilder` then constructs a comprehensive prompt consisting of a system preamble, conversation history, the retrieved RAG excerpts, and the user's question. Finally, the configured `LlmProvider` calls the underlying API (e.g., via the Google GenAI Java SDK [50]). The listing below shows the structured prompt assembly.

```
public String buildRuntimePrompt(String summary,
                                 List<Message> recentMessages,
                                 List<String> ragChunks,
                                 String userQuestion,
                                 boolean firstTurn) {
    StringBuilder sb = new StringBuilder();
    sb.append("You are a helpful academic teaching assistant for
    students\n");
    sb.append("using the StudySpace platform.\n\n");

    if (firstTurn) {
        // Ask the LLM to generate a conversation title on the first
    turn
        sb.append("IMPORTANT: Begin with TITLE: <5-8 word summary>\n\
    n");
    }

    if (summary != null && !summary.isBlank()) {
        sb.append("## Conversation Summary (long-term memory)\n");
        sb.append(summary.trim()).append("\n\n");
    }
    if (recentMessages != null && !recentMessages.isEmpty()) {
        sb.append("## Recent Conversation\n");
        for (Message msg : recentMessages) {
            String label = "user".equalsIgnoreCase(msg.getRole()) ? "
    Student" : "Assistant";
            sb.append(label).append(": ").append(msg.getContent()).
    append("\n");
        }
        sb.append("\n");
    }
    if (ragChunks != null && !ragChunks.isEmpty()) {
        sb.append("## Relevant Document Excerpts\n");
        for (int i = 0; i < ragChunks.size(); i++) {
            sb.append("### Excerpt ").append(i + 1).append("\n");
            sb.append(ragChunks.get(i).trim()).append("\n\n");
        }
    }
```

```
34      sb.append("## Student's Current Question\n").append(userQuestion.
        trim());
35      return sb.toString();
36  }
```

Code listing 4.5 Code listing 4.5 — Structured prompt construction (PromptBuilder.java)

Rather than injecting a fixed-size slice of the document, the system uses the pgvector similarity search to select the top-3 chunks that are most semantically relevant to the student's question. This targeted retrieval keeps the prompt compact while ensuring that the most pertinent passages are always present. When the API returns a `429 RESOURCE_EXHAUSTED` status, the service surfaces the retry delay to the user as a human-readable message rather than propagating a raw exception.

4.5 Access Control (FR-05)

The access control feature addresses FR-05: the system must distinguish Instructors from Students at the API level and reject unauthorised requests before they reach the service layer. The technical challenge is making the authentication filter stateless so that no server-side session store is required, while still providing the full user identity (including role) to every downstream service call.

The student dashboard interface demonstrating the role-restricted layout and navigation options is shown in Figure 4.7. The two navigation sidebars illustrate the role distinction. The student sees course browsing and workspace links; the Instructor additionally sees the course management and Merge Proposal review links. Both views are generated from the same component by reading the role field from the JWT stored in the React context.

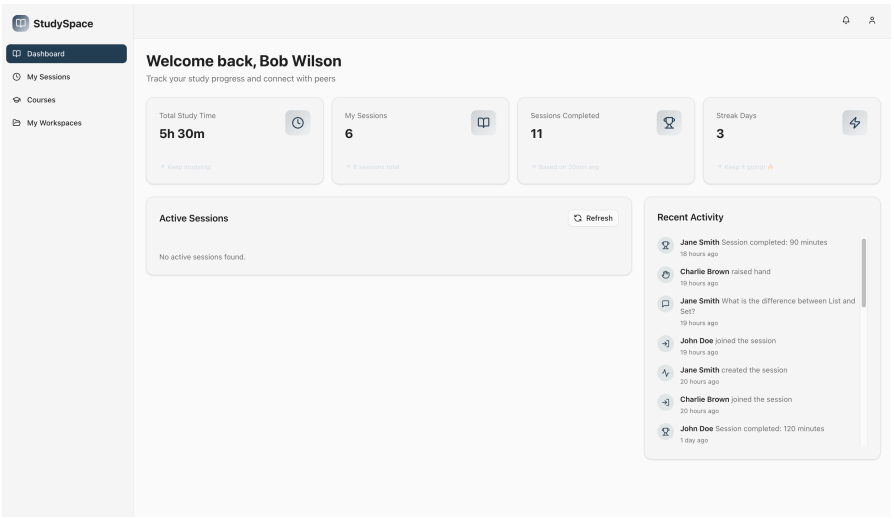

Figure 4.7 Student dashboard view illustrating the role-restricted navigation layout

When a user authenticates, `AuthService` validates the credentials, looks up the `User` entity, and calls `JwtUtil.generateToken` to produce a signed JWT embedding the user's email as the subject. The token is signed with HMAC-SHA256 and expires after ten hours. On every subsequent request, `JwtAuthenticationFilter` extracts the token from the `Authorization:Bearer` header, validates the signature and expiry, loads the `UserDetails` by email, and populates the Spring Security context. The listing below shows the core of the filter.

```
// Runs once per request; short-circuits if no token is present
String authHeader = request.getHeader("Authorization");
String jwt = null;

if (authHeader != null && authHeader.startsWith("Bearer ")) {
    jwt = authHeader.substring(7); // strip "Bearer " prefix
} else if (request.getParameter("token") != null) {
    jwt = request.getParameter("token"); // WebSocket fallback
}

if (jwt == null) { filterChain.doFilter(request, response); return; }

String userEmail = jwtUtil.extractUsername(jwt); // decode subject
    claim
if (userEmail != null
        && SecurityContextHolder.getContext().getAuthentication() ==
    null) {
    UserDetails userDetails = userDetailsService.loadUserByUsername(
    userEmail);
    if (jwtUtil.isTokenValid(jwt, userDetails)) {
```

```
18          // Populate security context so @PreAuthorize annotations
    work
19          UsernamePasswordAuthenticationToken authToken =
20              new UsernamePasswordAuthenticationToken(
21                  userDetails, null, userDetails.getAuthorities());
22          SecurityContextHolder.getContext().setAuthentication(
    authToken);
23      }
24  }
25  filterChain.doFilter(request, response);
```

Code listing 4.6 Code listing 4.6 — JWT validation in the authentication filter (JwtAuthenticationFilter.java)

The filter populates the Spring Security context with the user's authorities (which encode the role), enabling `@PreAuthorize` annotations and explicit `assertInstructor` checks in the service layer to reject Student requests on Instructor-only operations. The stateless approach means that the server holds no session state between requests, satisfying NFR-02 and simplifying horizontal scaling.

4.6 Testing

The application was verified through three complementary testing strategies. Automated tests validated the correctness of individual components and complete request-response cycles at the code level. Structured usability test scenarios were prepared to guide future evaluation with the target user group. The subsections below describe each layer in turn.

4.6.1 Backend Automated Tests

The backend uses three levels of automated tests, all implemented with JUnit 5 [54] and the Spring Boot Test framework.

Unit tests cover individual service classes in isolation. Dependencies are substituted with Mockito [55] mocks so that each test exercises exactly one unit. *Repository tests* verify that the Spring Data JPA repositories produce correct SQL and return expected entity graphs. They run against an in-process H2 database seeded with test data. `UserRepositoryTest` checks that user look-up by email and username behaves correctly.

Integration tests exercise complete request-response cycles through the Spring MVC layer. `UserControllerIntegrationTest` and `StudySessionControllerIntegrationTest` spin up a `@SpringBootTest` application context, issue HTTP requests via `MockMvc`, and assert on the HTTP status code and the JSON response body. `GlobalExceptionHandlerTest` verifies that the centralised exception handler maps domain exceptions to the correct HTTP status codes.

During development, several important bugs were found and corrected through testing. A referential-integrity violation when deleting reference materials was resolved by introducing the soft-delete `isHidden` flag described in section 4.2.

4.6.2 Frontend Automated Tests

Complementing the backend testing suite, the frontend employs a dual strategy to verify user interfaces and application flows.

Unit and integration tests are executed using Vitest to isolate components such as authentication forms. By mocking the Next.js router and authentication contexts, the tests simulate user input and assert on the resulting DOM updates without relying on a running server.

End-to-end (E2E) tests are orchestrated with Playwright to navigate complete user journeys in a headless browser. To verify all five core functional requirements (FR-01 to FR-05) under realistic conditions, a dedicated E2E test suite intercepts API calls and simulates user actions, ensuring that the frontend behaves correctly under various backend states (e.g., successful login or HTTP 401 Unauthorized). A representative test case verifying the course administration and browsing workflows is shown in code listing 4.7.

```
test('FR-01 - Browse Course Materials', async ({ page }) => {
  await page.route('**/api/courses?*', async route => {
    await route.fulfill({
      json: { content: [{ id: 101, title: 'Software Engineering
    Methodologies', description: 'Learn Scrum, XP, Agile.',
    instructorName: 'Dr. John Doe', isPublished: true }], totalPages:
     1, totalElements: 1 }
    });
  });
  await page.route('**/api/courses/101', async route => {
    await route.fulfill({
      json: { id: 101, title: 'Software Engineering Methodologies',
    sections: [{ id: 201, title: 'Week 1: Introduction to Scrum',
    materials: [{ id: 301, title: 'Scrum Guide PDF', type: 'PDF', url:
     '/mock/scrum-guide.pdf' }] }] }
    });
  });
  await page.goto('/courses');
  await expect(page.locator('text=Software Engineering Methodologies
    ')).toBeVisible();
  await page.getByRole('link', { name: /view course/i }).first().
    click();
  await expect(page.locator('text=Week 1: Introduction to Scrum')).
    toBeVisible();
  await expect(page.locator('text=Scrum Guide PDF')).toBeVisible();
});
```

Code listing 4.7 Playwright E2E test verifying course material browsing

Table 4.1 summarises the results of the E2E test suite execution, demonstrating full coverage across all functional areas.

Table 4.1 Playwright E2E test suite execution summary

Test ID	Target Feature Area	Status	Duration
FR-01	Course Administration (Browse Materials)	Passed	1.1 s
FR-02	Content Extension (Merge Proposal)	Passed	1.5 s
FR-03 & FR-04	Contextual Messaging & AI Querying	Passed	2.1 s
FR-05	Access Control (Private Routes)	Passed	0.8 s

4.6.3 User and Usability Tests

The following test scenarios were designed to validate the five functional requirements against the workflows described in the use cases in section 2.3. Each scenario is specified as a numbered step sequence with explicit pre-conditions and expected outcomes, so that it can be executed by a member of the target user group (a student or Instructor at a higher-education faculty) without technical assistance. Formal usability testing with representative participants has not been conducted within the scope of this thesis; these scenarios constitute the prepared protocol for that future evaluation.

Test 4.1: Upload and browse a Course Material (FR-01)

1. Log in as an Instructor account.
2. Navigate to an existing published course and open the course management view.
3. Select a section and click the upload button. Choose a PDF file of at least 1 MB.
4. Confirm the upload. Expected: the material appears in the section list with the correct file name and a PDF icon.
5. Log out. Log in as an enrolled Student. Navigate to the same course and section.
6. Confirm the material is visible and can be downloaded.

Edge case: Attempt the upload step (step 3) while logged in as a Student. Expected: the upload button is absent from the UI; a direct API call to `POST /api/courses/sections/id/materials` returns HTTP 403 Forbidden.

Test 4.2: Clone course and submit Merge Proposal (FR-02)

1. Log in as a Student enrolled in a published course.
2. Navigate to the course and click the “Clone to workspace” button.
3. Confirm the Private Workspace is created with sections and materials mirroring the original course.
4. Upload a new PDF into one of the workspace sections.
5. Select the uploaded material, click “Contribute”, choose an existing target section, optionally add a message, and submit the Merge Proposal.
6. Log out and log in as the course Instructor. Navigate to the Merge Proposal review tab.

7. Confirm the proposal appears with status `PENDING`. Approve it.
8. Navigate to the course section. Confirm the approved material appears with the contributor's name.

Edge case: In step 5, select a reference material (one cloned from the course) rather than a newly uploaded file. Expected: the proposal is accepted, but the backend copies the file from the original URL into the course storage, not from a non-existent student upload.

Test 4.3: Post a message with Contextual Anchor (FR-03)

1. Log in as a Student and open a workspace.
2. In the chat panel, type `@` followed by part of a Course Material name.
3. Confirm the autocomplete dropdown appears with matching materials.
4. Select a material. Confirm the chip is inserted inline in the message input.
5. Submit the message. Confirm the chip is rendered as a clickable badge in the chat.
6. Click the badge. Confirm the Course Material file is downloaded or opened.

Edge case: Open the same workspace as a second enrolled Student. Confirm the message with the Contextual Anchor is visible to the second student and the badge is functional.

Test 4.4: Query AI assistant (FR-04)

1. Log in as a Student, open a workspace, and select a Course Material (PDF).
2. Open the AI assistant panel and type a factual question about the document's content.
3. Submit. Confirm the response appears with a typewriter animation and references concepts from the document rather than generic knowledge.
4. Type a follow-up question. Confirm the AI response reflects the conversation history.

Edge case: Open the AI panel without selecting any Course Material and submit a question. Expected: the backend sends the question without document context and the AI responds with general knowledge; no error is shown.

Test 4.5: Role-based access control (FR-05)

1. Log in as a Student. Open the browser developer tools and copy the JWT from the `Authorization` header of any API request.

2. Using `curl` or Swagger UI, call `POST /api/courses` (Instructor-only endpoint) with the Student JWT.
3. Expected: the server returns HTTP 403 Forbidden with an error body.
4. Repeat for `PUT /api/courses/{id}/sections` and `POST /api/proposals/{id}/review`.
5. In each case confirm the response is 403 and the operation has no effect.

Edge case: Submit a request with a malformed or expired JWT to any protected endpoint. Expected: the server returns HTTP 401 Unauthorized with the message "Unauthorized".

4.7 Implementation Summary

The implementation chapter described the construction of StudySpace across five functional areas. The course administration module established the data model and a profile-switchable file storage backend (Local/S3). The content extension system realised the clone mechanism and Merge Proposal workflow. The contextual messaging system introduced the portable anchor token, allowing users to reference specific Course Materials in chat. The AI assistant integrated the Google Gemini API through a structured prompt pipeline to ground answers in the actual course document. The access control layer tied all features together, enforcing role separation at the API boundary.

Automated testing verified the correctness of the main flows across unit, repository, and integration levels. The application was deployed to Vercel (frontend) and Render (backend) through an automated GitLab CI/CD pipeline that enforces all checks before any code reaches production.

4.8 Project Organisation

This section describes the tooling and processes used to manage the codebase, automate quality checks and deployment, and maintain project documentation throughout the development lifecycle. The project is maintained in a single Git repository, adopting a monorepo layout that separates the `backend/`, `frontend/`, and `documentation/` directories. Version control is managed via Git, with changes committed directly to the main repository to facilitate rapid iteration in development stage.

4.8.1 CI/CD Pipeline

A GitLab [56] CI/CD pipeline runs automatically on every push on master branch. The pipeline defines multiple stages to guarantee code quality and automate deployment [57], as shown in Figure 4.8. The first stages build the

backend with Gradle [58] and the frontend with `npm`, followed by comprehensive unit, integration, and end-to-end (Playwright) testing. Subsequent stages run security and code quality scans, including OWASP Dependency-Check [59] and SonarQube [60]. If all checks pass on the main branch, the final stage allows manual triggering of the deployment jobs. Once triggered, the frontend is deployed to Vercel. A separate pipeline job builds the backend container, pushes it to Docker Hub, and triggers the deployment hook on the Render cloud platform.

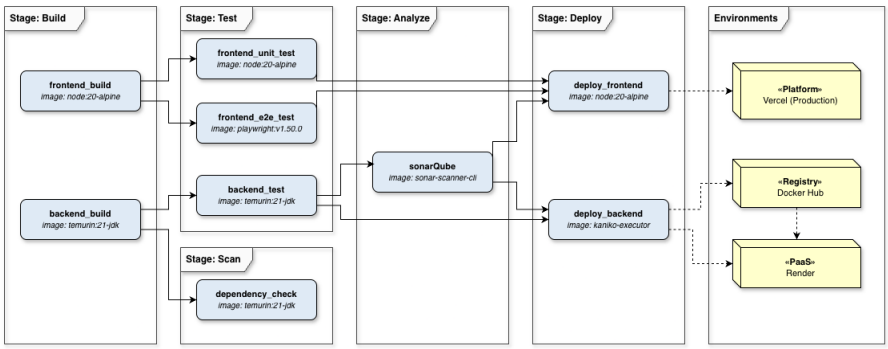

Figure 4.8 GitLab CI/CD pipeline stages and deployment workflow

The pipeline enforces that `master` is always in a deployable state. No code reaches the branch without passing the backend tests and the frontend type check, which together cover the most common categories of regression.

4.8.2 Documentation

Comprehensive documentation is essential to ensure that the StudySpace platform remains maintainable, extensible, and accessible to future developers and users. The project employs a multi-tiered documentation strategy, encompassing automated API references, codebase READMEs, and structured supplementary guides.

The backend API is documented automatically by Springdoc OpenAPI [14], which generates a Swagger UI [51] from the controller annotations and makes it available at `/swagger-ui/index.html` when the application is running. Public service methods carry JavaDoc [61] comments that explain the purpose and contract of each operation.

The repository contains three README files that serve as the primary entry points for developers and operators. The root-level `README.md` describes the project purpose, lists the live deployment URLs, and outlines the monorepo layout. The `backend/README.md` covers the core technology stack, the key architectural decisions, and the full set of environment variables required to run the server locally or in a containerised environment. The `frontend/README.md`

details the frontend stack, the key architectural patterns, and local setup instructions for the Next.js development server.

A separate user guide is provided as a supplementary document to walk through the main student and Instructor workflows with annotated screenshots. The developer guide is integrated directly into the repository across the three README files, which provide step-by-step instructions for setting up the full development environment, configuring the required environment variables, running the test suites, and deploying new versions.

Chapter 5
Conclusion

The main goal of this thesis was to solve the tool fragmentation experienced by higher-education students and instructors. Rather than treating document viewing, communication, and AI assistance as isolated processes, this project aimed to develop a unified web application, StudySpace, that bridges the gap between passive content consumption and active collaboration. The resulting system serves as a functional proof of concept demonstrating the viability of this unified approach.

The resulting application successfully fulfils all functional requirements established at the start of the project. It provides Instructors with a course administration module to manage content and enrollments. For students, the system introduces a content extension system, allowing them to clone courses into a Private Workspace and submit Merge Proposals. The application also features a contextual messaging system that anchors discussions directly to source documents, and an integrated AI assistant that provides answers grounded exclusively in the uploaded Course Materials.

By delivering these core capabilities within a secure, role-aware application, this thesis demonstrates that a unified workspace can effectively eliminate the need for disjointed learning and communication platforms.

5.1 Future Work

While the current deployment successfully establishes the core workflows, it leaves several directions open for subsequent development.

The Contextual Anchor currently stores only the material identifier and title. A natural extension would be to encode a page number or byte offset alongside the material reference, allowing the receiving client to open the PDF viewer at the precise location the sender was viewing at the time of writing.

The enrollment model supports two statuses (`ACTIVE` and `DROPPED`) but does not include an enrollment request or Instructor approval step. A real

faculty deployment would require an invitation or explicit approval workflow before granting student access to Course Materials in restricted courses.

Although the system already records cumulative study time and daily streak data per user, this information is currently surfaced only in individual profile and dashboard views. A practical extension would be to aggregate these metrics into a cohort-level engagement overview, giving students a sense of their own progress relative to peers and providing instructors with a lightweight tool for identifying students who may require additional support. Such a feature would make fuller use of the data the system already collects and could serve as a concrete motivational signal without introducing any change to the existing database schema.

Finally, the backend was deployed to a single-instance PaaS environment. Achieving true horizontal scalability under concurrent load would require two infrastructure upgrades: externalising the WebSocket session state through a shared STOMP broker (such as RabbitMQ or ActiveMQ), and provisioning the JWT signing secret through a dedicated secrets vault rather than a plain environment variable.

Appendix A

Supplementary Material

This appendix contains supplementary material that is not required to understand the main text.

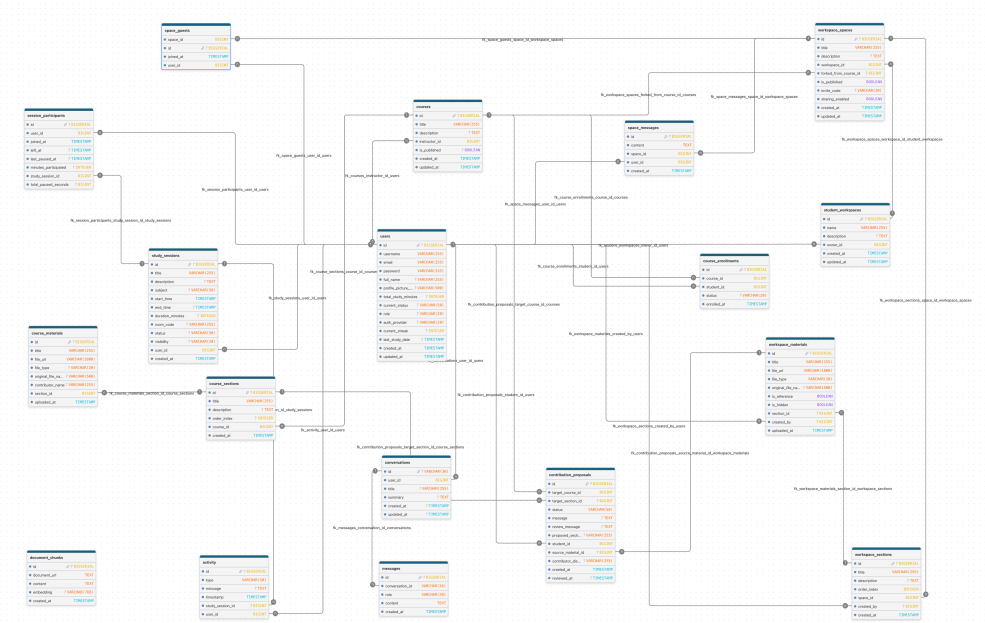

Figure A.1 Complete StudySpace domain model UML diagram

Bibliography

1. META PLATFORMS, INC. *React: A JavaScript library for building user interfaces* [online]. 2024. Available also from: `https://react.dev/`. [Accessed 5. 3. 2026].
2. EVAN YOU. *Vue.js: The Progressive JavaScript Framework* [online]. 2024. Available also from: `https://vuejs.org/`. [Accessed 5. 3. 2026].
3. GOOGLE LLC. *Angular: The web development framework for building the future* [online]. 2024. Available also from: `https://angular.dev/`. [Accessed 6. 3. 2026].
4. MICROSOFT CORPORATION. *TypeScript: Typed JavaScript at Any Scale* [online]. 2024. Available also from: `https://www.typescriptlang.org/`. [Accessed 28. 3. 2026].
5. VERCEL INC. *Next.js by Vercel: The React Framework* [online]. 2024. Available also from: `https://nextjs.org/`. [Accessed 7. 3. 2026].
6. TAILWIND LABS INC. *Tailwind CSS: Rapidly build modern websites without ever leaving your HTML* [online]. 2024. Available also from: `https://tailwindcss.com/`. [Accessed 8. 3. 2026].
7. SHADCN. *shadcn/ui: Beautifully designed components that you can copy and paste into your apps* [online]. 2024. Available also from: `https://ui.shadcn.com/`. [Accessed 9. 3. 2026].
8. META PLATFORMS, INC. *React Documentation: Passing Data Deeply with Context* [online]. 2024. Available also from: `https://react.dev/learn/passing-data-deeply-with-context`. [Accessed 10. 3. 2026].
9. ORACLE CORPORATION. *Java Platform, Standard Edition 21* [online]. 2023. Available also from: `https://docs.oracle.com/en/java/javase/21/`. [Accessed 3. 5. 2026].
10. BROADCOM INC. *Spring Boot* [online]. 2024. Available also from: `https://spring.io/projects/spring-boot`. [Accessed 11. 3. 2026].

11. BROADCOM INC. *Spring Security* [online]. 2024. Available also from: https://spring.io/projects/spring-security. [Accessed 11. 3. 2026].
12. BROADCOM INC. *Spring Data JPA: Reference Documentation* [online]. 2024. Available also from: https://docs.spring.io/spring-data/jpa/reference/. [Accessed 12. 3. 2026].
13. BROADCOM INC. *Spring Framework: WebSocket Support* [online]. 2024. Available also from: https://docs.spring.io/spring-framework/reference/web/websocket.html. [Accessed 12. 3. 2026].
14. SPRINGDOC.ORG. *Springdoc OpenAPI: OpenAPI 3 Library for Spring Boot* [online]. 2024. Available also from: https://springdoc.org/. [Accessed 13. 3. 2026].
15. OPENJDK. *JEP 444: Virtual Threads* [online]. 2023. Available also from: https://openjdk.org/jeps/444. [Accessed 13. 3. 2026].
16. OPENJS FOUNDATION. *Node.js* [online]. 2024. Available also from: https://nodejs.org/. [Accessed 14. 3. 2026].
17. KAMIL MYSLIWIEC. *NestJS: A progressive Node.js framework* [online]. 2024. Available also from: https://nestjs.com/. [Accessed 15. 3. 2026].
18. OPENJS FOUNDATION. *Express: Node.js web application framework* [online]. 2024. Available also from: https://expressjs.com/. [Accessed 15. 3. 2026].
19. OPENJS FOUNDATION. *About Node.js: Asynchronous event-driven JavaScript runtime* [online]. 2024. Available also from: https://nodejs.org/en/about. [Accessed 16. 3. 2026].
20. PYTHON SOFTWARE FOUNDATION. *Welcome to Python.org* [online]. 2024. Available also from: https://www.python.org/. [Accessed 16. 3. 2026].
21. DJANGO SOFTWARE FOUNDATION. *The Web framework for perfectionists with deadlines | Django* [online]. 2024. Available also from: https://www.djangoproject.com/. [Accessed 17. 3. 2026].
22. SEBASTIÁN RAMÍREZ. *FastAPI* [online]. 2024. Available also from: https://fastapi.tiangolo.com/. [Accessed 17. 3. 2026].
23. PYTHON SOFTWARE FOUNDATION. *What is the Python Global Interpreter Lock (GIL)?* [Online]. 2024. Available also from: https://wiki.python.org/moin/GlobalInterpreterLock. [Accessed 18. 3. 2026].
24. FOWLER, Martin. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2003. ISBN 0321127420.
25. RICHARDS, Mark. *Software Architecture Patterns*. 2nd. O'Reilly Media, Inc., 2022. ISBN 9781098134273.

26. THE POSTGRESQL GLOBAL DEVELOPMENT GROUP. *PostgreSQL: The World's Most Advanced Open Source Relational Database* [online]. 2024. Available also from: https://www.postgresql.org/. [Accessed 18. 3. 2026].

27. BROADCOM INC. *Spring Framework: Hibernate ORM Integration* [online]. 2024. Available also from: https://docs.spring.io/spring-framework/reference/data-access/orm/hibernate.html. [Accessed 28. 6. 2026].

28. MONGODB, INC. *MongoDB: The Developer Data Platform* [online]. 2024. Available also from: https://www.mongodb.com/. [Accessed 20. 3. 2026].

29. KATZ, Jonathan et al. *pgvector: Open-source vector similarity search for Postgres* [online]. 2024. Available also from: https://github.com/pgvector/pgvector. [Accessed 21. 3. 2026].

30. IBM CORPORATION. *What is a REST API?* [Online]. 2024. Available also from: https://www.ibm.com/think/topics/rest-apis. [Accessed 15. 5. 2026].

31. JONES, M.; BRADLEY, J.; SAKIMURA, N. *RFC 7519: JSON Web Token (JWT)* [online]. 2015. Available also from: https://datatracker.ietf.org/doc/html/rfc7519. [Accessed 21. 3. 2026].

32. BROADCOM INC. *Spring Session* [online]. 2024. Available also from: https://spring.io/projects/spring-session. [Accessed 22. 3. 2026].

33. IETF. *OAuth 2.0* [online]. 2012. Available also from: https://oauth.net/2/. [Accessed 22. 3. 2026].

34. FETTE, I.; MELNIKOV, A. *RFC 6455: The WebSocket Protocol* [online]. 2011. Available also from: https://datatracker.ietf.org/doc/html/rfc6455. [Accessed 6. 3. 2026].

35. STOMP WORKING GROUP. *STOMP Protocol Specification, Version 1.2* [online]. 2012. Available also from: https://stomp.github.io/stomp-specification-1.2.html. [Accessed 23. 3. 2026].

36. TEAM, SockJS. *SockJS-client* [online]. 2024. Available also from: https://github.com/sockjs/sockjs-client. [Accessed 23. 3. 2026].

37. WHATWG. *Server-Sent Events* [online]. 2024. Available also from: https://html.spec.whatwg.org/multipage/server-sent-events.html. [Accessed 24. 3. 2026].

38. LORETO, S.; SAINT-ANDRE, P.; SALSANO, S.; WILKINS, G. *RFC 6202: Known Issues and Best Practices for the Use of Long Polling and Streaming in Bidirectional HTTP* [online]. 2011. Available also from: https://datatracker.ietf.org/doc/html/rfc6202. [Accessed 25. 3. 2026].

39. META PLATFORMS, INC. *Jest: Delightful JavaScript Testing* [online]. 2024. Available also from: https://jestjs.io/. [Accessed 25. 3. 2026].

40. VITEST TEAM. *Vitest: A blazing fast unit test framework powered by Vite* [online]. 2024. Available also from: https://vitest.dev/. [Accessed 26. 3. 2026].

41. TESTING LIBRARY CONTRIBUTORS. *React Testing Library* [online]. 2024. Available also from: https://testing-library.com/docs/react-testing-library/intro/. [Accessed 4. 5. 2026].

42. CYPRESS.IO. *Cypress: JavaScript Component Testing and E2E Testing Framework* [online]. 2024. Available also from: https://www.cypress.io/. [Accessed 26. 3. 2026].

43. MICROSOFT CORPORATION. *Playwright: Fast and reliable end-to-end testing for modern web apps* [online]. 2024. Available also from: https://playwright.dev/. [Accessed 27. 3. 2026].

44. MICROSOFT CORPORATION. *Playwright: Parallelism and sharding* [online]. 2024. Available also from: https://playwright.dev/docs/test-parallel. [Accessed 28. 6. 2026].

45. DOCKER INC. *Docker Compose* [online]. 2024. Available also from: https://docs.docker.com/compose/. [Accessed 12. 3. 2026].

46. VERCEL INC. *Vercel: Develop. Preview. Ship.* [Online]. 2024. Available also from: https://vercel.com/. [Accessed 3. 4. 2026].

47. RENDER SERVICES, INC. *Render: Cloud Application Hosting for Developers* [online]. 2024. Available also from: https://render.com/. [Accessed 7. 4. 2026].

48. TIGRIS DATA, INC. *Tigris: Developer-focused object storage built on FoundationDB* [online]. 2024. Available also from: https://www.tigrisdata.com/. [Accessed 11. 4. 2026].

49. AMAZON WEB SERVICES, INC. *Amazon Simple Storage Service (S3)* [online]. 2024. Available also from: https://aws.amazon.com/s3/. [Accessed 16. 4. 2026].

50. GOOGLE LLC. *Google GenAI Java SDK* [online]. 2024. Available also from: https://github.com/googleapis/java-genai. [Accessed 20. 4. 2026].

51. SMARTBEAR SOFTWARE. *Swagger UI: Visualize and interact with the API's resources* [online]. 2024. Available also from: https://swagger.io/tools/swagger-ui/. [Accessed 25. 4. 2026].

52. WHATWG. *HTML Living Standard: The `contenteditable` content attribute* [online]. 2024. Available also from: https://html.spec.whatwg.org/multipage/interaction.html#contenteditable. [Accessed 28. 4. 2026].

53. APACHE SOFTWARE FOUNDATION. *Apache PDFBox: A Java PDF Library* [online]. 2024. Available also from: `https://pdfbox.apache.org/`. [Accessed 29. 4. 2026].

54. JUNIT TEAM. *JUnit 5 User Guide* [online]. 2024. Available also from: `https://junit.org/junit5/docs/current/user-guide/`. [Accessed 1. 5. 2026].

55. MOCKITO CONTRIBUTORS. *Mockito: Tasty mocking framework for unit tests in Java* [online]. 2024. Available also from: `https://site.mockito.org/`. [Accessed 2. 5. 2026].

56. GITLAB INC. *GitLab: The Most-Comprehensive AI-Powered DevSecOps Platform* [online]. 2024. Available also from: `https://about.gitlab.com/`. [Accessed 7. 5. 2026].

57. HUMBLE, Jez; FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley Professional, 2010. ISBN 9780321601919.

58. GRADLE, INC. *Gradle Build Tool* [online]. 2024. Available also from: `https://gradle.org/`. [Accessed 8. 5. 2026].

59. OWASP FOUNDATION. *OWASP Dependency-Check* [online]. 2024. Available also from: `https://owasp.org/www-project-dependency-check/`. [Accessed 10. 5. 2026].

60. SONARSOURCE SA. *SonarQube Server Documentation* [online]. 2024. Available also from: `https://docs.sonarsource.com/sonarqube/latest/`. [Accessed 28. 6. 2026].

61. ORACLE CORPORATION. *Javadoc Tool* [online]. 2024. Available also from: `https://docs.oracle.com/javase/8/docs/technotes/tools/windows/javadoc.html`. [Accessed 28. 6. 2026].

Contents of Enclosed Media

/
- `README.md` ... brief description of the repository and setup instructions
- `backend` ..... directory containing the backend Spring Boot application source code
- `frontend` . directory containing the frontend Next.js application source code
- `documentation` ..... directory containing the thesis documentation
  - `ctufit-thesis.pdf` ..... the final thesis text in PDF format
  - `user-guide.pdf` ..... the user guide for the application
- `docker-compose.yml` .... configuration file for running the application locally